République Tunisienne

Ministère de l'Enseignement Supérieur

et de la Recherche Scientifique

RAPPORT DE STAGE D'ÉTÉ

Stage Data Science — Durée : 2 mois

Construction d'un Système RAG pour l'Automatisation de la Recherche d'Information en Entreprise

Assistant documentaire interne à réponses sourcées

`LangChain` · `ChromaDB` · `Ollama / Mistral 7B`

`sentence-transformers` · `RAGAS`

`FastAPI` · `Streamlit` · `Docker`

Réalisé par : TLILI Mohamed Aziz

Encadrante académique : Mme ENNINE Hanene

Organisme d'accueil : ESPRIT-Tech

SPÉCIALITÉ : DATA SCIENCE

2025 / 2026

Dédicaces

Je dédie ce travail

À mes parents,
qui m'ont soutenu et encouragé tout au long de ce parcours.
Ce travail vous est entièrement dédié,
et j'espère vous rendre fiers.

À mon encadrante académique Mme ENNINE Hanene,
pour sa disponibilité, ses précieux conseils
et son accompagnement bienveillant durant toute cette période.

À tous ceux qui, de près ou de loin,
ont contribué à l'aboutissement de ce projet.

Remerciements

Au terme de ce stage, je tiens à exprimer ma sincère gratitude à toutes les personnes qui ont contribué, directement ou indirectement, à la réalisation de ce travail.

Je remercie tout particulièrement mon encadrante académique, Mme ENNINE Hanene, pour la confiance qu'elle m'a accordée, pour la clarté de ses orientations méthodologiques et pour la rigueur qu'elle a su m'inculquer tout au long de ce projet. Ses remarques sur la question centrale de l'évaluation de la fiabilité d'un système génératif ont profondément orienté la structure de ce travail.

Mes remerciements s'adressent également à toute l'équipe d'ESPRIT-Tech pour la qualité de l'environnement de travail mis à ma disposition et pour la disponibilité dont elle a fait preuve durant ces deux mois.

Je remercie enfin les communautés open source à l'origine des outils sur lesquels repose ce projet — LangChain, ChromaDB, Ollama, sentence-transformers, RAGAS, FastAPI et Streamlit — dont la documentation et les contributions rendent aujourd'hui accessible la construction de systèmes d'intelligence artificielle entièrement locaux et gratuits.

Table des matières

Dédicaces **iii**

Remerciements **iv**

Introduction Générale **1**

1 Contexte général **4**
1.1 Introduction 4
1.2 Présentation de l'organisme d'accueil 4
1.3 Cadre stratégique et valeurs de l'organisme 4
1.4 Méthodologie CRISP-DM 5
1.5 Les étapes de la méthodologie CRISP-DM 5
1.6 Conclusion 7

2 Compréhension métier **8**
2.1 Introduction 8
2.2 Problématique 8
2.2.1 Le fossé lexical 8
2.2.2 La charge de synthèse 8
2.2.3 L'hallucination des modèles génératifs 9
2.3 Étude de l'existant 9
2.4 Solution proposée 9
2.5 Critères de succès 11
2.6 Conclusion 11

3 Compréhension et préparation des données **13**
3.1 Collecte des données 13
3.2 Description des données 13
3.2.1 Composition du corpus 13
3.2.2 Analyse statistique 13
3.3 Exploration des données 15
3.3.1 Structure des documents 15
3.3.2 Nature transactionnelle du corpus et limite identifiée 15
3.4 Préparation des données 16

3.4.1 Chargement et normalisation 16
3.4.2 Découpage en segments 17
3.4.3 Effet réel du découpage sur ce corpus 17
3.4.4 Récapitulatif du pipeline d'ingestion 18
3.5 Conclusion 19

4 Modélisation : architecture du système RAG 20
4.1 Introduction 20
4.2 Architecture générale 20
4.3 Vectorisation sémantique 22
4.3.1 Principe 22
4.3.2 Choix du modèle 22
4.3.3 Normalisation 23
4.4 Indexation vectorielle 23
4.4.1 ChromaDB 23
4.4.2 Indexation par lots 24
4.5 Recherche sémantique 24
4.6 Génération contrainte 25
4.6.1 Le modèle de langue 25
4.6.2 L'invite système : premier levier anti-hallucination 25
4.6.3 Garde-fou sur le contexte vide 26
4.7 Orchestration avec LangChain 27
4.8 Synthèse des paramètres 28
4.9 Conclusion 28

5 Évaluation de la fiabilité du système 29
5.1 Introduction 29
5.2 Évaluation de la recherche sémantique 29
5.2.1 Protocole 29
5.2.2 Résultats 30
5.2.3 Diagnostic : l'asymétrie linguistique 30
5.2.4 Limite structurelle : la recherche sur identifiant exact 32
5.3 Évaluation de la chaîne complète : le protocole RAGAS 32
5.3.1 Les quatre métriques retenues 32
5.3.2 Mise en uvre 33
5.3.3 État d'avancement et résultats 34
5.4 Performance et latence 35
5.5 Leviers anti-hallucination : bilan 36
5.6 Conclusion 36

6 Déploiement **38**
6.1 Introduction 38
6.2 Architecture de déploiement 38
6.3 L'API REST avec FastAPI 39
6.4 L'interface conversationnelle 40
6.4.1 Écran d'accueil 40
6.4.2 Réponse sourcée 40
6.5 Conteneurisation avec Docker 41
6.5.1 Organisation 41
6.5.2 Mise en service 42
6.6 Points de vigilance avant mise en production 43
6.7 Versionnement 44
6.8 Conclusion 44

Conclusion Générale **46**

Bibliographie et Nétographie **50**

Table des figures

1.1 Le processus CRISP-DM et ses six phases itératives. Les flèches en pointillés représentent les retours en arrière fréquents, notamment entre modélisation et préparation des données. 7

4.1 Architecture du système RAG. La phase hors ligne construit l'index vectoriel ; la phase en ligne l'interroge à chaque question. Le modèle de vectorisation est nécessairement le même dans les deux phases. 21

6.1 Architecture de déploiement en trois conteneurs. Seul le port 8501 de l'interface est destiné aux utilisateurs ; l'API et le serveur LLM restent internes au réseau Docker. 38

6.2 Écran d'accueil de l'assistant. La barre latérale affiche l'état du système ; la zone centrale propose des questions d'amorçage cohérentes avec le corpus indexé. 41

6.3 Réponse sourcée produite par le système. Les quatre bons de commande sont restitués avec leurs détails, suivis de la liste des fichiers sources. La ligne de métadonnées indique le nombre d'extraits, le meilleur score et le temps de génération. 42

Liste des tableaux

2.1 Analyse comparative des solutions existantes 10
2.2 Critères de succès définis en phase de compréhension métier 12

3.1 Composition du corpus documentaire 14
3.2 Statistiques de longueur du texte extrait (en caractères) 14
3.3 Bilan du pipeline d'ingestion 18

4.1 Caractéristiques de l'index vectoriel construit 24
4.2 Paramètres du pipeline RAG 28

5.1 Scores de pertinence mesurés sur l'index de 2 929 segments 30
5.2 Expérience contrôlée — effet de la langue de la question sur le score de pertinence (index et corpus identiques) 31
5.3 Répartition des métriques RAGAS par composant évalué 33
5.4 Résultats RAGAS — à compléter après exécution de la campagne de mesure 35
5.5 Mesures de performance du système 35
5.6 Leviers de réduction des hallucinations 36

6.1 *Endpoints* de l'API REST 39
6.2 Services définis dans `docker-compose.yml` 41
6.3 Points de vigilance identifiés 43

Introduction Générale

Les organisations produisent et accumulent aujourd'hui des volumes de documents internes sans précédent : rapports d'activité, procédures, manuels techniques, contrats, factures, bons de commande, bases de connaissances. Ce patrimoine informationnel constitue une ressource stratégique, mais il reste largement sous-exploité. L'information existe, elle est archivée, mais elle demeure difficilement accessible : retrouver un chiffre précis ou une procédure particulière suppose de savoir dans quel document chercher, puis de parcourir manuellement ce document. Le coût de cette recherche croît linéairement avec la taille du corpus, jusqu'à devenir prohibitif.

Les moteurs de recherche documentaires classiques, fondés sur la correspondance de mots-clés, n'apportent qu'une réponse partielle à ce problème. Ils retournent une liste de documents, non une réponse : l'utilisateur doit encore lire et synthétiser. Surtout, ils échouent dès que la formulation de la question ne reprend pas le vocabulaire exact du document — une requête sur les ń congés ż ne trouvera pas un texte parlant d'ń absences autorisées ż.

L'émergence des grands modèles de langue (*Large Language Models*, LLM) a transformé cette situation. Ces modèles savent comprendre une question formulée en langage naturel et produire une réponse rédigée. Employés seuls, ils présentent toutefois deux limites disqualifiantes en contexte professionnel. D'abord ils ne connaissent pas les documents internes de l'entreprise, qui ne font pas partie de leurs données d'entraînement. Ensuite, et plus gravement, ils *hallucinent* : lorsqu'ils ignorent une réponse, ils en produisent une plausible mais fausse, sans signaler leur incertitude. Un assistant interne qui inventerait un montant de facture ou une clause contractuelle serait non seulement inutile, mais dangereux.

L'architecture *RAG* (*Retrieval-Augmented Generation*) répond précisément à ces deux limites. Son principe est d'ancrer la génération dans des documents réels : plutôt que d'interroger le modèle sur ses connaissances internes, on recherche d'abord les passages pertinents dans le corpus de l'entreprise, puis on demande au modèle de répondre *uniquement* à partir de ces passages, en citant ses sources. La connaissance reste dans les documents ; le modèle n'est plus qu'un moteur de reformulation contraint.

Objectifs du stage

Ce stage, d'une durée de 2 mois, a pour objet la conception, la réalisation et le déploiement d'un système RAG complet permettant d'interroger en langage naturel un corpus de documents d'entreprise. Cinq objectifs ont été fixés :

1. collecter, nettoyer et structurer les documents internes et mettre en place un pipeline d'indexation automatique ;
2. générer des *embeddings* sémantiques et les stocker dans une base de données vectorielle ;
3. connecter la recherche sémantique à un LLM via un *framework* d'orchestration afin de produire des réponses sourcées ;
4. évaluer la fiabilité du système et réduire les hallucinations à l'aide d'un *framework* d'évaluation dédié ;
5. déployer l'assistant via une API et une interface web dans un environnement conteneurisé.

Une contrainte transverse a structuré l'ensemble des choix techniques : l'intégralité de la chaîne devait fonctionner **localement et sans coût d'API**. Les documents internes d'une entreprise sont par nature confidentiels ; les transmettre à un service tiers pour vectorisation ou génération soulève des questions de conformité difficilement acceptables. Cette contrainte, loin d'être une limitation, s'est révélée être un argument central en faveur de l'architecture retenue.

Méthodologie et plan du rapport

Le projet a été conduit selon la méthodologie **CRISP-DM** (*Cross-Industry Standard Process for Data Mining*), qui structure un projet de science des données en six phases itératives. Le plan de ce rapport suit fidèlement cette progression.

Le **chapitre 1** présente l'organisme d'accueil et détaille la méthodologie CRISP-DM. Le **chapitre 2** formalise la problématique métier, analyse les solutions existantes et justifie la solution retenue. Le **chapitre 3** décrit le corpus documentaire, son exploration et sa préparation. Le **chapitre 4** expose l'architecture technique du système RAG : vectorisation, indexation, recherche sémantique et génération contrainte. Le **chapitre 5** est consacré à l'évaluation de la fiabilité du système et aux leviers mis en place contre les hallucinations. Le **chapitre 6** présente le déploiement : API REST, interface web et conteneurisation. Une conclusion générale dresse le bilan du travail réalisé et ouvre sur des perspectives

d'amélioration.

CHAPITRE 1

Contexte général

1.1 Introduction

Ce chapitre situe le cadre dans lequel le projet a été réalisé. Nous présentons d'abord l'organisme d'accueil, sa vision et ses valeurs. Nous exposons ensuite la méthodologie de conduite de projet retenue — CRISP-DM — en détaillant ses six phases et en précisant la manière dont chacune s'applique à un projet de type RAG, qui présente la particularité de ne pas reposer sur l'entraînement d'un modèle mais sur l'assemblage et le réglage d'une chaîne de composants.

1.2 Présentation de l'organisme d'accueil

Créée en 2010, ESPRIT-Tech est la structure dédiée à la Recherche, au Développement et à l'Innovation (RDI) au sein de l'école ESPRIT. Depuis la mise en place des programmes de formation, l'activité RDI est devenue l'une des priorités stratégiques de l'établissement.

ESPRIT-Tech regroupe des enseignants-chercheurs et offre un soutien complet en matière de financement, de formation et d'accompagnement dans les projets de recherche. La structure définit les stratégies de recherche en collaboration avec les différentes parties prenantes, en mettant particulièrement l'accent sur l'innovation et la recherche appliquée. C'est dans ce cadre que le présent projet a été mené, à l'intersection du traitement automatique du langage naturel et de l'ingénierie logicielle.

1.3 Cadre stratégique et valeurs de l'organisme

ESPRIT-Tech a pour vision de promouvoir la recherche, l'innovation et la valorisation des résultats scientifiques. Elle s'engage à soutenir l'excellence en RDI à travers le développement de projets à fort impact, répondant aux besoins académiques et industriels.

Sa mission consiste à créer un environnement propice à la recherche et à l'innovation, à renforcer le financement externe et à développer un écosystème entrepreneurial ouvert

à l'international, favorisant la collaboration entre chercheurs et partenaires.

Les valeurs de ESPRIT-Tech reposent sur la rigueur scientifique, le respect, l'inclusion et la transparence. Cette exigence de transparence a d'ailleurs eu une traduction technique directe dans le présent projet : le système développé ne se contente pas de produire une réponse, il expose systématiquement les extraits documentaires sur lesquels cette réponse s'appuie, avec leur score de pertinence. L'utilisateur peut ainsi vérifier ce que le système avance plutôt que de devoir lui faire confiance.

1.4 Méthodologie CRISP-DM

CRISP-DM, ou *Cross-Industry Standard Process for Data Mining*, est un modèle de processus qui permet d'orienter les travaux d'exploration de données en science des données. Cette méthodologie itérative permet de travailler en cycles : de la compréhension du besoin métier jusqu'au déploiement, en passant par la préparation des données, la modélisation et l'évaluation. Elle donne un cadre structuré pour guider le *data scientist* tout au long du cycle de vie d'un projet.

Son choix pour ce stage se justifie par deux caractéristiques. D'une part, son caractère **itératif** : dans un projet RAG, la qualité finale dépend de paramètres — taille des segments, nombre de documents récupérés, formulation de l'invite — dont la valeur optimale ne peut être déterminée *a priori*. Il faut mesurer, ajuster, remesurer. D'autre part, la place centrale qu'elle accorde à la phase d'**évaluation** avant tout déploiement : c'est précisément le point critique d'un système génératif, dont la sortie est du texte en langage naturel et non une classe ou une valeur numérique aisément comparable à une vérité de terrain.

1.5 Les étapes de la méthodologie CRISP-DM

La méthodologie CRISP-DM se compose de six étapes clés.

Étape 1 : la compréhension du besoin métier

Dans cette phase initiale, on travaille en collaboration avec les parties prenantes afin de comprendre les objectifs, les exigences et les contraintes du projet ainsi que les bénéfices attendus. Les ressources nécessaires — budget, compétences techniques, accès aux données — sont évaluées. Les problèmes à résoudre sont identifiés et les critères permettant de mesurer le succès du projet sont définis. Pour ce projet, cette étape a produit la contrainte structurante du ń tout local ż et le critère de succès central : la traçabilité de chaque réponse. Elle fait l'objet du chapitre 2.

Étape 2 : la compréhension des données

Il s'agit de collecter les données disponibles, d'en décrire la structure, d'en évaluer la qualité et d'identifier les premières tendances. Dans un projet RAG, cette étape porte sur le corpus documentaire : quels types de documents, quel volume, quelle qualité d'extraction textuelle, quelle structure interne. Elle détermine directement la stratégie de découpage adoptée ensuite.

Étape 3 : la préparation des données

Phase généralement la plus coûteuse en temps, elle consiste à nettoyer, transformer et mettre en forme les données pour l'étape suivante. Ici, elle recouvre l'extraction du texte des fichiers PDF, l'enrichissement en métadonnées de traçabilité et le découpage en segments de taille compatible avec le modèle de vectorisation. Les étapes 2 et 3 constituent le chapitre 3.

Étape 4 : la modélisation

On construit des modèles prédictifs ou descriptifs en fonction des objectifs du projet. Plusieurs itérations ont lieu entre les étapes de préparation et de modélisation pour affiner l'usage de certains algorithmes.

Cette étape mérite une remarque particulière dans le cadre d'un projet RAG. Contrairement à un projet d'apprentissage supervisé classique, aucun modèle n'est ici *entraîné* : le modèle de vectorisation et le modèle de langue sont tous deux pré-entraînés et utilisés en inférence. La ń modélisation ż consiste à **assembler et paramétrer une chaîne de traitement** : choix du modèle d'*embeddings*, structure de l'index vectoriel, stratégie de recherche, formulation de l'invite système. Ce sont ces choix, et non des poids appris, qui déterminent la performance du système. C'est l'objet du chapitre 4.

Étape 5 : l'évaluation

Une fois les modèles construits, ils sont évalués afin de déterminer leur qualité. Cette étape permet de s'assurer que le système atteint les objectifs du projet. Les performances sont mesurées à l'aide de métriques appropriées et comparées aux critères de succès définis en première phase. Si les résultats ne répondent pas aux attentes, les étapes précédentes peuvent être révisées et répétées.

Pour un système génératif, l'évaluation pose une difficulté propre : il n'existe pas de métrique unique comme l'*accuracy*. Nous verrons au chapitre 5 comment le *framework* RAGAS décompose la qualité d'un système RAG en quatre dimensions mesurables séparément, dont une — la *faithfulness* — mesure directement le taux d'hallucination.

Étape 6 : le déploiement

Enfin, les résultats du projet sont présentés aux parties prenantes et intégrés si nécessaire dans les systèmes existants pour aider à la prise de décision. Cette phase implique la création de rapports, de visualisations ou d'autres formes de communication rendant les résultats compréhensibles et utilisables par des non-spécialistes. Elle correspond au chapitre 6 : API REST, interface conversationnelle et conteneurisation.

La figure 1.1 illustre le caractère cyclique de ce processus.

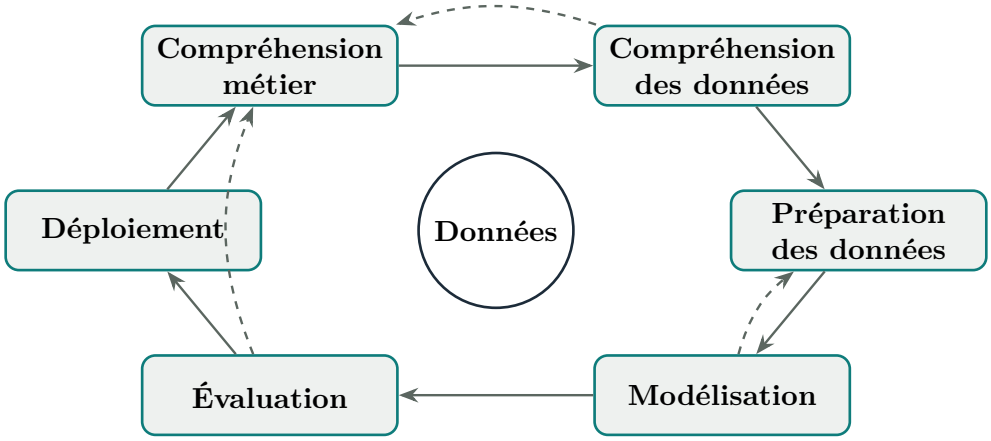


Figure 1.1 – Le processus CRISP-DM et ses six phases itératives. Les flèches en pointillés représentent les retours en arrière fréquents, notamment entre modélisation et préparation des données.

1.6 Conclusion

Ce chapitre a présenté ESPRIT-Tech, son rôle, sa vision et ses valeurs, ainsi que la méthodologie CRISP-DM retenue pour structurer le projet. Nous avons souligné les deux adaptations que requiert cette méthodologie dans un contexte RAG : la modélisation devient un travail d'assemblage et de paramétrage plutôt que d'entraînement, et l'évaluation exige des métriques spécifiques aux systèmes génératifs. Le chapitre suivant formalise la problématique métier à laquelle le système doit répondre.

CHAPITRE 2

Compréhension métier

2.1 Introduction

Ce chapitre formalise le besoin auquel le système doit répondre. Nous caractérisons d'abord la problématique de l'accès à l'information documentaire en entreprise, puis nous analysons les solutions existantes et leurs limites. Nous exposons enfin la solution proposée et les critères objectifs retenus pour juger de son succès.

2.2 Problématique

Une entreprise de taille moyenne accumule, sur quelques années d'activité, plusieurs dizaines de milliers de documents hétérogènes. Cette accumulation crée un paradoxe : l'information est présente et conservée, mais son coût d'accès la rend pratiquement inutilisable. Trois obstacles distincts se combinent.

2.2.1 Le fossé lexical

Les moteurs de recherche documentaires classiques reposent sur la correspondance de mots-clés. Une requête ne trouve un document que si les termes employés par l'utilisateur figurent littéralement dans ce document. Or les rédacteurs et les lecteurs n'emploient pas le même vocabulaire : une question portant sur les ń congés ż ne remontera pas une procédure intitulée ń absences autorisées ż, et une recherche sur ń fournisseur ż ignorera un document parlant de ń prestataire ż. Ce phénomène, connu sous le nom de *fossé lexical* (*vocabulary mismatch*), constitue la limite structurelle de la recherche par mots-clés.

2.2.2 La charge de synthèse

Même lorsqu'un moteur classique retrouve les bons documents, il retourne une *liste de documents*, non une *réponse*. L'utilisateur doit encore ouvrir chaque résultat, localiser le passage utile et synthétiser mentalement. Pour une question dont la réponse est répartie sur plusieurs documents, ce travail est long et sujet à l'oubli.

2.2.3 L'hallucination des modèles génératifs

Le recours direct à un modèle de langue généraliste, tel que ChatGPT, semble apporter une réponse immédiate à ces deux obstacles : il comprend le langage naturel et produit une réponse rédigée. Deux difficultés majeures l'interdisent cependant en contexte professionnel.

D'abord, le modèle **ne connaît pas les documents de l'entreprise**. Ils n'appartiennent pas à ses données d'entraînement, et ils ne le peuvent pas : transmettre des contrats, des factures ou des données salariales à un service tiers pose un problème de confidentialité et de conformité réglementaire (RGPD) rédhibitoire pour la plupart des organisations.

Ensuite, et plus gravement, le modèle **hallucine**. Lorsqu'il ne dispose pas de l'information demandée, il ne répond pas ń je ne sais pas ż : il génère la suite de texte statistiquement la plus vraisemblable, c'est-à-dire une réponse plausible dans sa forme mais fausse dans son contenu. Un assistant interne qui inventerait un montant de facture, une date d'échéance contractuelle ou une étape de procédure de sécurité serait pire qu'inutile : il serait dangereux, d'autant que sa formulation assurée n'offre à l'utilisateur aucun signal d'alerte.

> **Formulation de la problématique.** Comment permettre à un collaborateur d'interroger en langage naturel l'ensemble des documents internes de son entreprise, en obtenant une réponse directe, *vérifiable* et *traçable*, sans que ces documents ne quittent l'infrastructure de l'organisation ?

2.3 Étude de l'existant

Plusieurs familles de solutions adressent tout ou partie de cette problématique. Le tableau 2.1 en présente une analyse comparative.

Cette analyse fait ressortir un constat : aucune solution du marché ne réunit simultanément la confidentialité totale, l'absence de coût récurrent et la traçabilité vérifiable des réponses. C'est cet espace que le projet vise à occuper.

2.4 Solution proposée

Nous proposons de construire un assistant documentaire interne fondé sur l'architecture **RAG** (*Retrieval-Augmented Generation*), dont le principe est de découpler la *connaissance* de la *formulation*.

Table 2.1 – Analyse comparative des solutions existantes

Solution	**Avantages**	**Inconvénients**
Recherche plein texte (Elasticsearch, Solr)	Mature, très rapide, passe à l'échelle sur des millions de documents ; aucun coût d'inférence	Sensible au fossé lexical ; retourne des documents et non des réponses ; la charge de synthèse reste entièrement sur l'utilisateur
LLM généraliste seul (ChatGPT, GPT-4o)	Excellente compréhension du langage naturel ; réponse directe et bien rédigée ; aucune infrastructure à maintenir	N'a aucune connaissance des documents internes ; hallucine sans le signaler ; envoi de données confidentielles à un tiers ; coût d'API récurrent
Suites propriétaires (Microsoft 365 Copilot, Glean)	Intégration native à l'écosystème bureautique ; RAG déjà implémenté ; maintenance déléguée	Coût par utilisateur élevé ; suppose que les documents soient déjà hébergés chez l'éditeur ; fonctionnement opaque, métriques de fiabilité non exposées ; dépendance forte au fournisseur
Assistants de base de connaissances (Notion AI, Confluence AI)	Simples d'usage ; bien intégrés à l'outil hôte	Limités aux documents de la plateforme ; ne couvrent pas les PDF et archives externes ; traçabilité des sources partielle
RAG sur mesure (LangChain, ChromaDB, Ollama)	**Fonctionne entièrement en local** ; aucune donnée ne sort de l'infrastructure ; aucun coût d'API ; sources citées et vérifiables ; métriques de fiabilité mesurables ; contrôle total sur chaque composant	Nécessite un développement et une maintenance internes ; latence supérieure sur matériel sans GPU ; qualité dépendante du réglage du pipeline

La connaissance reste dans les documents de l'entreprise, indexés dans une base vectorielle locale. Lorsqu'une question est posée, le système commence par rechercher les passages documentaires les plus proches sémantiquement de cette question. Ces passages sont ensuite injectés dans l'invite envoyée au modèle de langue, accompagnés d'une consigne stricte : répondre uniquement à partir du contexte fourni, et déclarer explicitement son ignorance si la réponse ne s'y trouve pas. Le modèle n'est plus une source de connaissance, mais un moteur de reformulation contraint.

Ce basculement change la nature du problème de l'hallucination. Le modèle ne peut plus inventer à partir de son savoir général, puisque l'invite lui interdit de sortir du contexte fourni ; et lorsqu'il le fait malgré tout, la présence des extraits sources permet à l'utilisateur de le détecter immédiatement.

Les choix technologiques découlent directement de la contrainte de confidentialité :

— **ChromaDB** comme base vectorielle, persistée sur disque local, plutôt qu'un service géré comme Pinecone ;
— **sentence-transformers** pour la vectorisation, exécuté en local sur CPU, plutôt que l'API `text-embedding-ada-002` d'OpenAI ;
— **Ollama** avec le modèle **Mistral 7B** pour la génération, plutôt que GPT-4o ;
— **LangChain** pour l'orchestration de la chaîne ;
— **RAGAS** pour la mesure de la qualité et du taux d'hallucination.

Aucun de ces composants n'émet de requête vers un service externe en phase d'exploitation. L'ensemble de la chaîne, de l'ingestion à la génération, s'exécute sur la machine hôte.

2.5 Critères de succès

Conformément à la première phase de CRISP-DM, des critères objectifs ont été définis en amont pour juger de la réussite du projet. Ils sont présentés au tableau 2.2 et repris au chapitre 5.

2.6 Conclusion

Ce chapitre a formalisé la problématique du projet en identifiant ses trois composantes : le fossé lexical qui limite la recherche par mots-clés, la charge de synthèse laissée à l'utilisateur, et le risque d'hallucination des modèles génératifs employés seuls. L'étude de l'existant a montré qu'aucune solution du marché ne concilie confidentialité, gratuité et

Table 2.2 – Critères de succès définis en phase de compréhension métier

Critère	**Description**	**Mesure**
Ancrage documentaire	Toute affirmation de la réponse doit être soutenue par le contexte récupéré	*Faithfulness* (RAGAS)
Pertinence de la réponse	La réponse doit adresser la question effectivement posée	*Answer relevancy*
Qualité de la recherche	Les extraits récupérés doivent être pertinents et complets	*Context precision* et *recall*
Traçabilité	Chaque réponse doit exposer ses documents sources et leur score	Vérification fonctionnelle
Confidentialité	Aucune donnée ne doit quitter l'infrastructure	Audit des dépendances réseau
Aveu d'ignorance	Le système doit refuser de répondre hors contexte	Test sur questions hors corpus

traçabilité. La solution proposée — une architecture RAG entièrement locale — répond à ces trois exigences. Le chapitre suivant s'intéresse au corpus documentaire sur lequel ce système a été construit.

CHAPITRE 3

Compréhension et préparation des données

3.1 Collecte des données

Le corpus utilisé provient du jeu de données public *Company Documents Dataset*, disponible sur la plateforme Kaggle. Il rassemble des documents d'entreprise générés à partir de la base de démonstration *Northwind*, un jeu de données commercial classique décrivant l'activité d'un distributeur agroalimentaire fictif : commandes, clients, produits, fournisseurs et employés.

Ce corpus a été retenu pour trois raisons. Il est **librement redistribuable**, ce qui autorise sa publication dans un dépôt de code. Il est **réaliste** dans sa structure : documents hétérogènes, générés automatiquement à partir d'un système d'information, tels qu'on en trouve effectivement dans les archives d'une entreprise. Il est enfin d'un **volume significatif** — plusieurs milliers de fichiers — permettant de valider le passage à l'échelle du pipeline d'ingestion.

Les fichiers, tous au format PDF, ont été déposés dans le répertoire `data/raw/` du projet, qui constitue le point d'entrée unique du pipeline d'ingestion.

3.2 Description des données

3.2.1 Composition du corpus

Le corpus rassemble **2 676 fichiers PDF** répartis en quatre familles documentaires, présentées au tableau 3.1.

3.2.2 Analyse statistique

Une analyse systématique de la longueur du texte extrait a été menée sur l'intégralité du corpus. Ses résultats figurent au tableau 3.2.

Table 3.1 – Composition du corpus documentaire

Famille	Fichiers	Part	Contenu
Factures	830	31,0 %	Identifiant de commande, coordonnées client, lignes de produits, montant total
Bons de commande	830	31,0 %	Identifiant, date, nom du client, produits commandés
Commandes	809	30,2 %	Détails d'expédition, client, employé, transporteur, lignes de produits
Rapports de stock	207	7,8 %	État mensuel des stocks par catégorie et par produit
Total	**2 676**	**100 %**	

Table 3.2 – Statistiques de longueur du texte extrait (en caractères)

Famille	Fichiers	Moyenne	Médiane	Min	Max	Pages
Factures	830	387	384	315	1 029	1,00
Bons de commande	830	210	207	152	820	1,00
Commandes	809	911	856	609	1 494	1,84
Rapports de stock	207	475	302	141	2 592	1,00
Ensemble	**2 676**	**498**	—	141	2 592	1,25

Trois enseignements se dégagent de ces mesures.

Le corpus est de volume textuel modeste. L'ensemble représente 1 332 652 caractères, soit environ 1,3 Mo de texte brut pour 2 676 fichiers. La taille moyenne d'un document est de 498 caractères, ce qui correspond à un paragraphe.

L'extraction textuelle est intégralement réussie. Aucun des 2 676 fichiers n'a produit un texte vide. Cette observation est importante : elle signifie que tous les PDF du corpus sont *nativement textuels* et non des images numérisées. Aucune reconnaissance optique de caractères (OCR) n'a donc été nécessaire, ce qui aurait considérablement alourdi le pipeline. Cette propriété favorable ne serait vraisemblablement pas vérifiée sur un corpus d'entreprise réel, qui comporte souvent des documents scannés.

La dispersion des longueurs est forte entre familles. Un bon de commande compte en moyenne 210 caractères, une commande 911, soit un rapport de un à quatre. Les rapports de stock présentent la plus forte variance (médiane 302, maximum 2 592) : certains mois ne comptent que quelques références en stock, d'autres plusieurs dizaines. Cette hétérogénéité a une conséquence directe sur la stratégie de découpage, analysée à la section 3.4.2.

3.3 Exploration des données

3.3.1 Structure des documents

L'examen du texte extrait révèle que les quatre familles suivent chacune un gabarit régulier, hérité de leur génération automatique. Le listing 3.1 présente une facture représentative.

Listing 3.1 – Texte extrait d'une facture (`invoice_10248.pdf`)

```
Invoice
Order ID: 10248
Customer ID: VINET
Order Date: 2016-07-04
Customer Details:
Contact Name: Paul Henriot
Address: 59 rue de l'Abbaye
City: Reims
Postal Code: 51100
Country: France
Phone: 26.47.15.10
Product Details:
Product ID Product Name Quantity Unit Price
11 Queso Cabrales 12 14.0
42 Singaporean Hokkien Fried Mee 10 9.8
72 Mozzarella di Giovanni 5 34.8
  TotalPrice 440.0
```

Un rapport de stock adopte une structure tabulaire différente, où chaque cellule est extraite sur une ligne distincte :

Listing 3.2 – Extrait d'un rapport de stock (`StockReport_2016-07.pdf`)

```
Stock Report for 2016-07
Category / Product / Units Sold / Units in Stock / Unit Price
Beverages      Chang                 105   17   19
Beverages      Chartreuse verte       48   69   18
Condiments     Chef Anton's Gumbo Mix 77    0    21.35
Confections    Pavlova                95   29   17.45
```

3.3.2 Nature transactionnelle du corpus et limite identifiée

L'exploration met en évidence une caractéristique déterminante du corpus, qu'il importe d'exposer clairement car elle conditionne l'interprétation des résultats du chapitre 5.

Les 2 676 documents sont exclusivement de nature transactionnelle. Il s'agit d'enregistrements structurés — factures, bons de commande, commandes, états de stock — c'est-à-dire, fonctionnellement, d'une base de données relationnelle exportée sous forme de fichiers PDF. Le corpus ne contient *aucune* procédure, *aucun* manuel technique, *aucune* note d'organisation ni base de connaissances rédigée.

Cette composition a une conséquence sur le type de questions auxquelles le système peut répondre. Il traite naturellement les interrogations factuelles portant sur un enregistrement précis — ń quels produits contient la commande 10590 ? ż, ń quel est le montant total de la facture 10248 ? ż — mais il ne peut évidemment pas répondre à une question procédurale telle que ń quelle est la procédure de remboursement des notes de frais ? ż, faute de document la décrivant.

Analyse critique. Une architecture RAG donne son plein rendement sur des documents *rédigés*, où l'information est diffuse dans un texte suivi et où la recherche sémantique apporte une valeur que la recherche exacte ne peut fournir. Sur des enregistrements structurés, une requête SQL sur la base d'origine serait plus rapide, plus exacte et moins coûteuse. Le corpus retenu permet donc de *valider techniquement* l'ensemble du pipeline — ingestion, vectorisation, recherche, génération, traçabilité — mais il sous-exploite l'intérêt propre de l'approche. L'enrichissement du corpus par des documents rédigés constitue la première perspective d'amélioration retenue en conclusion.

3.4 Préparation des données

3.4.1 Chargement et normalisation

Le module de chargement prend en charge cinq formats — PDF, DOCX, TXT, MD et CSV — et les normalise en objets `Document` de LangChain. Bien que le corpus actuel soit exclusivement au format PDF, cette polyvalence a été implémentée dès l'origine afin que le système accepte sans modification un corpus d'entreprise réel, nécessairement hétérogène.

Chaque document est enrichi de quatre métadonnées, dont dépend directement la capacité du système à citer ses sources :

Listing 3.3 – Enrichissement en métadonnées (`src/ingestion/loader.py`)

```
metadata = {
    "source":      str(path),                          # chemin complet
    "filename":    path.name,                          # nom affiche a l'utilisateur
    "file_type":   ext.lstrip("."),                    # pdf, docx, txt...
    "ingested_at": datetime.now(timezone.utc).isoformat(),
```

```
    }
    return Document(page_content=content, metadata=metadata)
```

Le chargement est *tolérant aux pannes* : un fichier illisible ou dépourvu de texte extractible est journalisé puis ignoré, sans interrompre le traitement des 2 675 autres. Sur un corpus de cette taille, l'échec global sur un seul fichier défectueux serait inacceptable en exploitation.

3.4.2 Découpage en segments

Le découpage (*chunking*) est l'opération qui divise un document en segments de taille compatible avec la fenêtre de contexte du modèle de vectorisation. Il est réalisé par un `RecursiveCharacterTextSplitter` paramétré à 1 000 caractères par segment et 150 caractères de chevauchement.

Ce découpeur est dit *récursif* car il tente de couper aux frontières naturelles du texte, par ordre de préférence décroissant : d'abord aux sauts de paragraphe (`\n\n`), puis aux sauts de ligne, puis aux fins de phrase, puis aux espaces, et en dernier recours à un caractère arbitraire. L'objectif est de préserver au maximum l'unité sémantique de chaque segment.

Le chevauchement de 150 caractères garantit qu'une information située à la frontière de deux segments reste intégralement présente dans au moins l'un des deux. Sans lui, une ligne de produit coupée en son milieu deviendrait irrécupérable pour la recherche.

Chaque segment reçoit enfin un identifiant lisible reliant le segment à son document d'origine, indispensable à la traçabilité :

Listing 3.4 – Attribution d'un identifiant de segment (`src/ingestion/chunker.py`)

```
counters: dict[str, int] = {}
for chunk in chunks:
    src = chunk.metadata.get("source", "unknown")
    counters[src] = counters.get(src, 0) + 1
    chunk.metadata["chunk_id"] = (
        f"{chunk.metadata.get('filename', src)}::chunk_{counters[src]}"
    )
```

Un segment est ainsi identifié par une chaîne du type `invoice_10248.pdf::chunk_1`, directement affichable dans l'interface.

3.4.3 Effet réel du découpage sur ce corpus

L'ingestion des 2 676 documents a produit **2 929 segments**, soit un ratio de 1,09 segment par document. Ce chiffre appelle une analyse.

La longueur moyenne d'un document étant de 498 caractères pour un paramètre `chunk_size` de 1 000, **la grande majorité des documents tient intégralement dans un seul segment**. Seuls 253 segments supplémentaires ont été produits, correspondant essentiellement aux documents de la famille ń Commandes ż — dont la longueur moyenne de 911 caractères et le maximum de 1 494 dépassent le seuil — ainsi qu'aux rapports de stock les plus fournis.

> **Conséquence méthodologique.** Sur ce corpus, le paramètre `chunk_size` est *quasiment inopérant* : le réglage du découpage, qui constitue habituellement l'un des principaux leviers d'optimisation d'un système RAG, n'a ici pratiquement aucun effet mesurable. Chaque segment correspond en pratique à un document entier. Cette situation présente un avantage — le contexte transmis au modèle est toujours cohérent et complet, jamais tronqué — mais elle prive l'étude d'un axe d'optimisation. Sur un corpus de documents rédigés, où un manuel de cinquante pages produirait plusieurs centaines de segments, ce paramètre redeviendrait déterminant.

3.4.4 Récapitulatif du pipeline d'ingestion

Le tableau 3.3 résume la transformation opérée.

Table 3.3 – Bilan du pipeline d'ingestion

Étape	Résultat
Fichiers présents dans `data/raw/`	2 676 PDF
Fichiers chargés avec succès	2 676 (100 %)
Fichiers sans texte extractible	0
Volume textuel total	1 332 652 caractères
Longueur moyenne par document	498 caractères
Segments produits après découpage	2 929
Ratio segments / document	1,09
Segments indexés dans ChromaDB	2 929

L'ingestion complète est déclenchée par une commande unique, ou par appel à l'*endpoint* `POST /ingest` de l'API :

Listing 3.5 – Lancement de l'ingestion

```
python scripts/ingest_documents.py --source data/raw --reset
```

L'option `--reset` supprime la collection existante avant ré-indexation, garantissant l'absence de doublons lors des ingestions successives.

3.5 Conclusion

Ce chapitre a décrit le corpus de 2 676 documents, son exploration statistique et sa préparation. Nous avons établi que l'extraction textuelle est intégralement réussie, que le volume total reste modeste (1,3 Mo de texte), et que le découpage produit 2 929 segments. Deux limites ont été identifiées et documentées : la nature exclusivement transactionnelle du corpus, qui sous-exploite l'intérêt propre d'une architecture RAG, et le caractère quasi inopérant du paramètre de découpage sur des documents aussi courts. Ces constats seront repris lors de l'interprétation des résultats. Le chapitre suivant présente l'architecture technique construite sur cette base.

CHAPITRE 4

Modélisation : architecture du système RAG

4.1 Introduction

Ce chapitre présente le cur technique du projet. Comme souligné au chapitre 1, la phase de modélisation ne consiste pas ici à entraîner un modèle, mais à **assembler et paramétrer une chaîne de traitement** dont chaque maillon influence la qualité finale. Nous détaillons successivement les quatre composants de cette chaîne : la vectorisation des segments, leur indexation dans une base vectorielle, la recherche sémantique, et la génération contrainte de la réponse. Nous exposons enfin la manière dont LangChain orchestre l'ensemble.

4.2 Architecture générale

Le système repose sur une séparation stricte entre deux phases, illustrée à la figure 4.1.

La **phase hors ligne** (ingestion) est exécutée une fois, puis à chaque évolution du corpus. Elle transforme les documents bruts en un index vectoriel persistant. Son coût est élevé mais amorti.

La **phase en ligne** (interrogation) est exécutée à chaque question. Elle ne touche jamais aux fichiers d'origine : elle interroge l'index, construit une invite et appelle le modèle de langue.

Ce découplage est une décision d'architecture importante. Il permet de ré-indexer le corpus sans interrompre le service, et de faire évoluer la stratégie de recherche ou l'invite système sans avoir à re-vectoriser les 2 929 segments — opération qui prend plusieurs minutes sur CPU.

Un point mérite d'être souligné sur cette figure : le modèle de vectorisation apparaît *deux fois*, et il doit impérativement s'agir du même. Les vecteurs des segments et celui

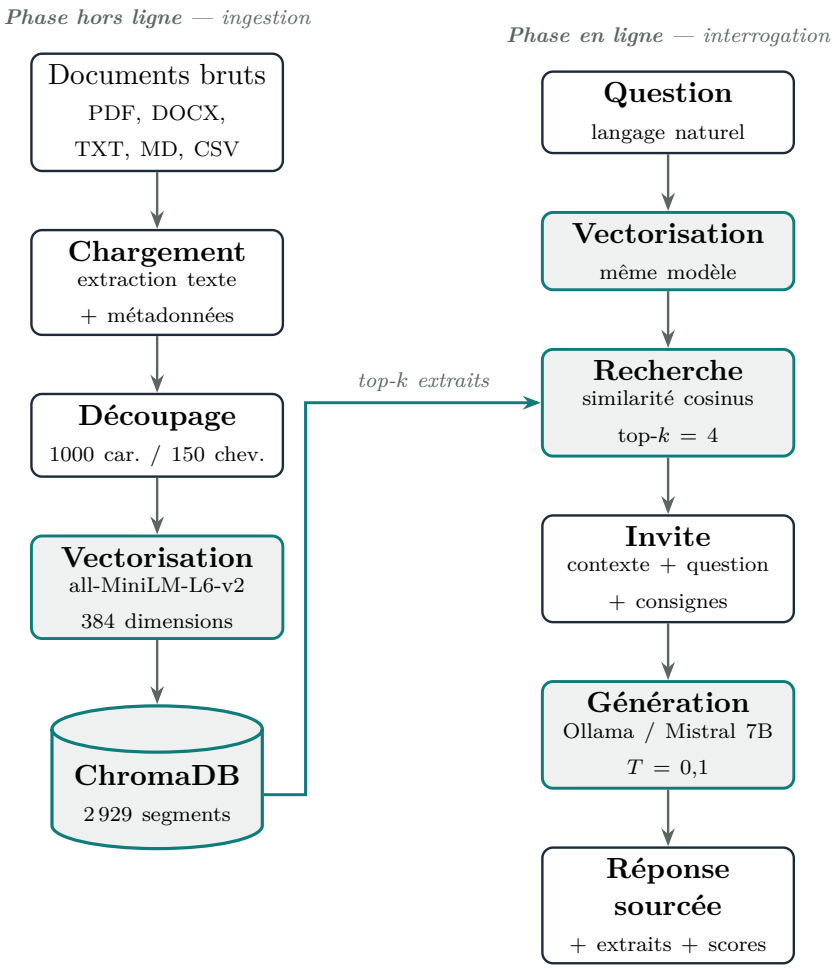


Figure 4.1 – Architecture du système RAG. La phase hors ligne construit l'index vectoriel ; la phase en ligne l'interroge à chaque question. Le modèle de vectorisation est nécessairement le même dans les deux phases.

de la question doivent appartenir au même espace sémantique pour que leur comparaison ait un sens. Changer de modèle d'*embeddings* impose donc de reconstruire intégralement l'index — une contrainte opérationnelle dont il faut tenir compte avant tout changement.

4.3 Vectorisation sémantique

4.3.1 Principe

La vectorisation, ou *embedding*, consiste à projeter un texte dans un espace vectoriel de dimension fixe, de telle sorte que la proximité géométrique entre deux vecteurs traduise la proximité de sens entre les deux textes. C'est ce mécanisme qui résout le fossé lexical identifié au chapitre 2 : ń congés ż et ń absences autorisées ż n'ont aucun caractère en commun, mais leurs vecteurs sont voisins.

Formellement, le modèle réalise une application

$$E : \mathcal{T} \longrightarrow \mathbb{R}^d$$

où $\mathcal{T}$ est l'ensemble des textes et d la dimension de l'espace d'arrivée. Pour le modèle retenu, $d = 384$.

4.3.2 Choix du modèle

Le modèle `sentence-transformers/all-MiniLM-L6-v2` a été retenu. Il s'agit d'une architecture Transformer à 6 couches, distillée depuis un modèle plus volumineux, comportant environ 22,7 millions de paramètres et produisant des vecteurs de dimension 384.

Ce choix résulte d'un compromis explicite entre trois exigences. La **gratuité et la confidentialité** d'abord : le modèle s'exécute localement, sans appel réseau, contrairement à l'API `text-embedding-ada-002` d'OpenAI mentionnée dans le sujet. La **légèreté** ensuite : avec 384 dimensions contre 1 536 pour le modèle d'OpenAI, l'index occupe quatre fois moins d'espace et la recherche est plus rapide ; le modèle tourne confortablement sur CPU. La **qualité** enfin : ce modèle constitue une référence établie pour la recherche sémantique en anglais, langue dans laquelle le corpus est rédigé.

> **Limite identifiée.** Le modèle `all-MiniLM-L6-v2` est majoritairement entraîné sur de l'anglais. Or les questions des utilisateurs sont formulées en français. Cette asymétrie dégrade la qualité de l'appariement lorsque la question et le document n'utilisent pas la même langue. Le passage à `paraphrase-multilingual-MiniLM-L12-v2`, entraîné sur plus de cinquante langues, constitue la première piste d'amélioration identifiée (*cf.*

chapitre 5). Ce changement impose toutefois une ré-indexation complète du corpus.

4.3.3 Normalisation

Les vecteurs sont normalisés à la production ($\|v\|_2 = 1$). Cette normalisation présente un avantage pratique : la similarité cosinus se réduit alors au simple produit scalaire,

$$\cos(u, v) = \frac{u \cdot v}{\|u\| \, \|v\|} = u \cdot v,$$

ce qui accélère la recherche et rend les scores directement comparables entre requêtes.

Listing 4.1 – Instanciation du modèle de vectorisation (`src/ingestion/indexer.py`)

```python
def get_embeddings() -> HuggingFaceEmbeddings:
    """Instancie paresseusement le modele d'embeddings (charge une fois,
    reutilise)."""
    global _embeddings_instance
    if _embeddings_instance is None:
        _embeddings_instance = HuggingFaceEmbeddings(
            model_name=settings.embedding_model,
            encode_kwargs={"normalize_embeddings": True},
        )
    return _embeddings_instance
```

Le chargement est *paresseux* et mis en cache dans une variable de module : le modèle, qui pèse environ 90 Mo, n'est chargé qu'au premier appel puis réutilisé. Sans cette précaution, chaque requête HTTP rechargerait le modèle depuis le disque, ajoutant plusieurs secondes de latence.

4.4 Indexation vectorielle

4.4.1 ChromaDB

Les vecteurs produits sont persistés dans une collection ChromaDB nommée `company_documents`, stockée localement dans `data/processed/chroma_db/`. ChromaDB a été préférée à Pinecone et Qdrant pour trois raisons : elle fonctionne intégralement en local sans serveur ni clé d'API, elle s'intègre nativement à LangChain, et ses performances sont largement suffisantes à l'échelle de plusieurs centaines de milliers de segments.

En interne, ChromaDB indexe les vecteurs à l'aide d'un graphe **HNSW** (*Hierarchical Navigable Small World*), une structure de recherche approximative de plus proches voisins. Elle évite le parcours exhaustif de la collection : la complexité de recherche passe de $O(n)$ à $O(\log n)$ environ, au prix d'une approximation en pratique négligeable.

4.4.2 Indexation par lots

L'insertion s'effectue par lots de 128 segments plutôt que segment par segment. Ce choix réduit sensiblement la durée totale d'indexation, la vectorisation bénéficiant du traitement vectoriel par lot, et il limite l'empreinte mémoire en évitant de maintenir l'intégralité des vecteurs en RAM.

Listing 4.2 – Indexation par lots (`src/ingestion/indexer.py`)

```python
def index_documents(chunks: list[Document], batch_size: int = 128) -> int:
    """Vectorise et persiste les segments dans ChromaDB."""
    if not chunks:
        return 0
    vectorstore = get_vectorstore()
    total = 0
    for i in range(0, len(chunks), batch_size):
        batch = chunks[i : i + batch_size]
        vectorstore.add_documents(batch)
        total += len(batch)
    return total
```

L'état final de l'index est présenté au tableau 4.1.

Table 4.1 – Caractéristiques de l'index vectoriel construit

Paramètre	**Valeur**
Base vectorielle	ChromaDB (persistée sur disque local)
Nom de la collection	`company_documents`
Nombre de segments indexés	2 929
Modèle de vectorisation	`all-MiniLM-L6-v2`
Dimension des vecteurs	384
Métrique de similarité	Cosinus (vecteurs normalisés)
Structure d'index	HNSW
Taille sur disque	environ 18 Mo
Taille de lot à l'insertion	128 segments

4.5 Recherche sémantique

À l'interrogation, la question est vectorisée par le même modèle, puis comparée à l'ensemble des vecteurs de l'index. Les k segments les plus proches sont retournés, accompagnés de leur score de pertinence.

Le paramètre k a été fixé à **4**. Sa valeur résulte d'un arbitrage : un k trop faible

risque d'omettre l'information nécessaire (dégradant le *context recall*) ; un k trop élevé introduit des segments hors sujet qui diluent le contexte, dégradent le *context precision* et augmentent le risque que le modèle s'appuie sur un passage non pertinent. La valeur $k = 4$ constitue par ailleurs un compromis de latence : chaque segment supplémentaire allonge l'invite et donc le temps de génération.

Listing 4.3 – Recherche avec scores (`src/rag/retriever.py`)

```python
def retrieve_with_scores(query: str, k: int | None = None):
    """Recupere les segments avec leur score de pertinence."""
    vectorstore = get_vectorstore()
    return vectorstore.similarity_search_with_relevance_scores(
        query, k=k or settings.top_k
    )
```

L'exposition des scores n'est pas un simple élément de confort de mise au point. Elle remplit deux fonctions dans le système final : elle alimente l'affichage de traçabilité présenté à l'utilisateur, et elle sert de signal pour le garde-fou décrit à la section 4.6.3.

4.6 Génération contrainte

4.6.1 Le modèle de langue

La génération est assurée par **Mistral 7B**, servi localement par **Ollama**. Le modèle, quantifié, occupe environ 4,4 Go sur disque et s'exécute sur CPU. Ollama expose une API HTTP locale sur le port 11434, à laquelle LangChain se connecte via le connecteur `ChatOllama`.

La **température** est fixée à **0,1**, valeur volontairement très basse. La température contrôle l'aléa de l'échantillonnage des *tokens* : plus elle est élevée, plus le modèle produit un texte varié et créatif. Or la créativité est exactement ce qu'il faut proscrire ici. Nous attendons du modèle qu'il restitue fidèlement le contenu du contexte fourni, non qu'il le reformule librement. Une température quasi nulle rend en outre les réponses largement reproductibles, ce qui est indispensable pour que l'évaluation du chapitre 5 soit stable d'une exécution à l'autre.

4.6.2 L'invite système : premier levier anti-hallucination

L'invite système constitue le principal levier de réduction des hallucinations au niveau de la génération. Elle a été formulée pour contraindre explicitement le modèle sur quatre points : la restriction au contexte fourni, l'obligation d'avouer son ignorance, l'interdiction d'inventer des faits, et la citation des sources.

Listing 4.4 – Invite système (`src/rag/pipeline.py`)

```
Tu es un assistant interne d'entreprise. Reponds UNIQUEMENT a partir du
contexte fourni ci-dessous, extrait des documents internes de l'entreprise.

Regles strictes :
- Si la reponse ne se trouve pas dans le contexte, reponds explicitement :
  "Je ne trouve pas cette information dans les documents disponibles."
- Ne jamais inventer de faits, chiffres ou procedures qui ne sont pas
  dans le contexte.
- Cite les documents sources utilises (nom de fichier) a la fin de ta reponse.
- Reponds de facon concise et professionnelle, en francais, sauf si la
  question est posee en anglais.

Contexte :
{context}

Question : {question}

Reponse :
```

Deux détails de formulation méritent d'être relevés. D'une part, la consigne d'ignorance fournit au modèle une *formulation exacte* à reprendre plutôt qu'une simple interdiction : lui donner une sortie de secours explicite est nettement plus efficace que de lui interdire d'inventer sans lui indiquer quoi faire à la place. D'autre part, chaque extrait injecté dans le contexte est préfixé du nom de son fichier d'origine, ce qui permet au modèle de citer correctement ses sources :

Listing 4.5 – Formatage du contexte avec attribution de source

```
def _format_docs(docs) -> str:
    return "\n\n---\n\n".join(
        f"[Source: {d.metadata.get('filename', 'unknown')}]\n{d.page_content}"
        for d in docs
    )
```

4.6.3 Garde-fou sur le contexte vide

L'invite ne suffit pas. Un second garde-fou, de nature programmatique, court-circuite entièrement le modèle lorsque la recherche ne remonte aucun segment. Dans ce cas, la réponse d'ignorance est retournée directement, sans appel au LLM.

Listing 4.6 – Garde-fou programmatique (`src/rag/pipeline.py`)

```
scored_docs = retrieve_with_scores(question, k=k)
```

```
if not scored_docs:
    return RAGResponse(
        answer="Je ne trouve pas cette information dans les documents disponibles.",
        sources=[],
        context_found=False,
    )
```

Ce garde-fou est plus robuste que l'invite, car il ne dépend pas de la docilité du modèle : un LLM peut ignorer une consigne, il ne peut pas contourner une instruction conditionnelle. C'est un principe de conception qu'il convient de généraliser : *tout ce qui peut être garanti par du code ne doit pas être délégué à l'invite.*

4.7 Orchestration avec LangChain

L'ensemble est assemblé au moyen du langage d'expression de chaînes de LangChain (*LangChain Expression Language*), qui permet de composer les étapes à l'aide de l'opérateur `|`.

Listing 4.7 – Composition de la chaîne RAG (`src/rag/pipeline.py`)

```
chain = (
    {"context": lambda x: _format_docs(docs), "question": RunnablePassthrough()}
    | _prompt
    | get_llm()
    | StrOutputParser()
)

answer = chain.invoke(question)
```

La chaîne se lit de gauche à droite : construction du dictionnaire de variables, application du gabarit d'invite, appel du modèle, extraction de la chaîne de caractères de sortie. Cette écriture déclarative rend la structure du pipeline immédiatement lisible et facilite le remplacement d'un maillon — changer de modèle de langue ne demande de modifier qu'un seul appel.

La réponse finale agrège le texte généré et les métadonnées de traçabilité :

Listing 4.8 – Construction de la réponse sourcée

```
sources = [
    {
        "filename": d.metadata.get("filename"),
        "chunk_id": d.metadata.get("chunk_id"),
        "relevance_score": round(float(score), 3),
    }
```

```
    for d, score in scored_docs
]

return RAGResponse(answer=answer, sources=sources, context_found=True)
```

4.8 Synthèse des paramètres

Le tableau 4.2 récapitule l'ensemble des paramètres du pipeline. Tous sont exposés par variables d'environnement, ce qui permet de les faire varier sans modification du code — condition nécessaire à l'itération mesurée décrite au chapitre suivant.

Table 4.2 – Paramètres du pipeline RAG

Paramètre	Valeur	Justification
`CHUNK_SIZE`	1000 caractères	Assez large pour conserver un document transactionnel entier, assez court pour rester sous la limite du modèle d'*embeddings*
`CHUNK_OVERLAP`	150 caractères	Évite qu'une information soit coupée à la frontière de deux segments
`TOP_K`	4	Compromis entre *recall* (ne rien manquer) et *precision* (ne pas diluer le contexte)
`EMBEDDING_MODEL`	`all-MiniLM-L6-v2`	Local, léger (384 dim.), gratuit
`OLLAMA_MODEL`	`mistral`	7 milliards de paramètres, exécutable sur CPU, licence ouverte
`OLLAMA_TEMPERATURE`	0,1	Minimise la créativité et rend les réponses reproductibles

4.9 Conclusion

Ce chapitre a détaillé l'architecture du système : vectorisation par `all-MiniLM-L6-v2`, indexation HNSW dans ChromaDB, recherche par similarité cosinus à $k = 4$, et génération contrainte par Mistral 7B à température 0,1. Nous avons insisté sur les deux leviers anti-hallucination mis en place — une invite système explicite et un garde-fou programmatique sur le contexte vide — ainsi que sur le principe qui les sous-tend : privilégier la garantie par le code sur la consigne en langage naturel. Reste à vérifier que ces leviers sont effectivement efficaces. C'est l'objet du chapitre suivant.

CHAPITRE 5

Évaluation de la fiabilité du système

5.1 Introduction

L'évaluation constitue la cinquième phase de CRISP-DM et, dans un projet RAG, la plus délicate. Un modèle de classification se juge à son *accuracy* : on compare une classe prédite à une classe attendue. Un système génératif produit du texte libre, pour lequel il n'existe pas de comparaison exacte à une vérité de terrain — deux formulations différentes peuvent être toutes deux correctes.

Ce chapitre présente donc une évaluation à deux niveaux. Nous exposons d'abord les mesures **quantitatives de la composante de recherche**, obtenues expérimentalement, qui ont mis en évidence une limitation majeure du système et conduit à une préconisation étayée. Nous présentons ensuite le **protocole d'évaluation RAGAS** de la chaîne complète, ainsi que les métriques retenues et l'état d'avancement de cette mesure.

5.2 Évaluation de la recherche sémantique

5.2.1 Protocole

La qualité de la recherche a été mesurée indépendamment de la génération. Cette séparation est méthodologiquement importante : si la recherche échoue à remonter le bon document, aucune qualité de génération ne peut compenser cette défaillance. Le contexte transmis au modèle sera erroné, et la réponse le sera également — fût-elle parfaitement fidèle au contexte reçu.

Sept questions ont été soumises au système, réparties en deux catégories : quatre portant sur des informations effectivement présentes dans le corpus, et trois portant sur des informations absentes (procédures RH, notes de frais, intégration des nouveaux employés). Pour chacune, nous avons relevé le score de pertinence maximal, le score minimal parmi les quatre extraits retournés, ainsi que la latence de recherche.

5.2.2 Résultats

Les mesures sont consignées au tableau 5.1.

Table 5.1 – Scores de pertinence mesurés sur l'index de 2 929 segments

Question	Score max	Score min	Latence
Informations présentes dans le corpus			
Quels produits contient la commande 10248 ?	0,082	0,077	193 ms
Quel est le montant total de la facture 10248 ?	0,115	0,099	160 ms
Quelles boissons étaient en stock en juillet 2016 ?	0,194	0,163	202 ms
`purchase orders de Maria Larsson`	0,567	0,550	148 ms
Informations absentes du corpus			
Quelle est la politique de congés payés ?	0,046	0,027	263 ms
Comment soumettre une note de frais ?	0,003	−0,022	151 ms
Quelle est la procédure d'intégration ?	−0,048	−0,068	158 ms

Deux constats se dégagent immédiatement.

Le premier est favorable : la **latence de recherche est très faible**, 182 ms en moyenne sur les sept requêtes. L'index HNSW de ChromaDB remplit parfaitement son office ; la recherche n'est en rien le goulot d'étranglement du système.

Le second est préoccupant : les scores des questions *présentes* dans le corpus sont anormalement bas. Trois des quatre questions légitimes obtiennent un score maximal inférieur à 0,20, à peine supérieur aux scores des questions *hors corpus* qui, elles, oscillent autour de 0. Un écart aussi faible entre les deux catégories interdit tout seuillage fiable : aucune valeur de seuil ne permettrait de rejeter les questions hors corpus sans rejeter simultanément la majorité des questions légitimes.

5.2.3 Diagnostic : l'asymétrie linguistique

L'examen des documents effectivement retournés révèle l'origine du problème. La question ń Quels produits contient la commande 10248 ? ż remonte les fichiers `invoice_10729.pdf` et `invoice_10840.pdf` — et non `invoice_10248.pdf`. La seule requête à obtenir un score élevé (0,567) est aussi la seule formulée majoritairement en anglais.

L'hypothèse est donc que le modèle `all-MiniLM-L6-v2`, entraîné essentiellement sur de l'anglais, projette mal les questions françaises dans l'espace sémantique des documents anglais. Pour la vérifier, une expérience contrôlée a été conduite : cinq questions ont été

formulées deux fois, une fois en français et une fois en anglais, avec un contenu sémantique identique, puis soumises au *même* index. Les résultats sont consignés au tableau 5.2.

Table 5.2 – Expérience contrôlée — effet de la langue de la question sur le score de pertinence (index et corpus identiques)

Contenu de la question	**Score FR**	**Score EN**	**Gain**
Produits de la commande 10248	0,082	0,337	+0,255
Montant total de la facture 10248	0,115	0,424	+0,309
Boissons en stock en juillet 2016	0,194	0,629	+0,435
Bons de commande de Maria Larsson	0,205	0,555	+0,350
Produits commandés par Paul Henriot	0,169	0,359	+0,190
Moyenne	**0,153**	**0,461**	+0,308 (×3,0)

Le résultat est sans ambiguïté : à contenu sémantique constant, la simple traduction de la question en anglais **triple le score de pertinence** (0,461 contre 0,153 en moyenne). L'effet est également qualitatif : la question sur les boissons en stock, formulée en anglais, remonte correctement un rapport de stock (`StockReport_2017-07_1.pdf`) alors que sa version française remontait un bon de commande, document d'un type entièrement inadapté.

Conclusion du diagnostic. La cause de la faiblesse des scores n'est ni le corpus, ni le découpage, ni le paramètre k, mais le **choix du modèle de vectorisation**. Un modèle monolingue anglais est structurellement inadapté à un usage où les questions sont posées en français sur des documents anglais. La préconisation qui en découle est de remplacer `all-MiniLM-L6-v2` par `paraphrase-multilingual-MiniLM-L12-v2`, entraîné sur plus de cinquante langues et conçu précisément pour aligner les formulations translinguistiques dans un espace commun. Ce remplacement suppose une ré-indexation complète du corpus, le changement de modèle invalidant l'intégralité des vecteurs existants.

Ce résultat illustre l'intérêt de la démarche d'évaluation quantitative : la faiblesse identifiée n'était pas décelable à l'usage. Les réponses produites paraissaient acceptables ; seule la mesure des scores a révélé que le système s'appuyait fréquemment sur les mauvais documents.

5.2.4 Limite structurelle : la recherche sur identifiant exact

Une seconde limite, indépendante de la langue, est apparue lors de ces mesures. Même en anglais, une question portant sur un identifiant précis (ń `order 10248` ż) ne remonte pas systématiquement le document portant cet identifiant.

Ce comportement est inhérent à la recherche vectorielle. Les *embeddings* encodent le *sens* d'un texte, non ses chaînes de caractères exactes. Or les identifiants numériques `10248`, `10249` et `10729` sont sémantiquement quasi indiscernables : ce sont tous des numéros de commande dans un contexte identique. Leur proximité vectorielle est donc très élevée, et le classement entre eux devient essentiellement arbitraire.

La solution reconnue pour ce type de besoin est la **recherche hybride**, combinant une composante lexicale exacte (BM25) et la composante vectorielle, puis fusionnant les deux classements. La composante lexicale trouve `10248` de manière exacte ; la composante vectorielle apporte la compréhension du sens. Cette amélioration figure parmi les perspectives retenues en conclusion.

5.3 Évaluation de la chaîne complète : le protocole RAGAS

5.3.1 Les quatre métriques retenues

L'évaluation de la chaîne complète repose sur le *framework* **RAGAS** (*Retrieval-Augmented Generation Assessment*), qui décompose la qualité d'un système RAG en dimensions mesurables séparément. Son principe est d'employer un modèle de langue comme *juge* : il compare la réponse produite, le contexte récupéré et la réponse de référence. Quatre métriques ont été retenues.

***Faithfulness* (fidélité).** C'est la métrique centrale du projet : elle mesure directement le taux d'hallucination. La réponse est décomposée en affirmations élémentaires, et l'on vérifie combien d'entre elles sont effectivement soutenues par le contexte récupéré :

$$\text{Faithfulness} = \frac{|\text{affirmations soutenues par le contexte}|}{|\text{affirmations totales de la réponse}|}$$

Un score de 1 signifie que rien n'a été inventé. Un score de 0,7 signifie que 30 % des affirmations produites ne figurent pas dans les documents — c'est-à-dire sont des hallucinations.

***Answer relevancy* (pertinence de la réponse).** Elle mesure si la réponse adresse effectivement la question posée. Une réponse peut être parfaitement fidèle au contexte tout en étant hors sujet. La métrique procède par génération inverse : à partir de la réponse, le juge produit des questions plausibles, puis mesure leur similarité cosinus moyenne avec la question d'origine :

$$\text{Answer relevancy} = \frac{1}{n}\sum_{i=1}^{n} \cos(E(q),\, E(q_i'))$$

où q est la question initiale et q_i' les questions régénérées.

***Context precision* (précision du contexte).** Elle évalue la composante de recherche : parmi les segments récupérés, quelle proportion est réellement pertinente ? Elle pénalise le bruit — les extraits hors sujet qui diluent le contexte et augmentent le risque que le modèle s'appuie sur un passage inadapté.

***Context recall* (rappel du contexte).** Elle évalue la complétude de la recherche : l'information nécessaire pour répondre a-t-elle été intégralement retrouvée ? C'est la seule des quatre métriques qui exige une réponse de référence rédigée par un humain, puisqu'il faut connaître l'information attendue pour juger si elle a été retrouvée.

Ces quatre métriques se répartissent naturellement en deux couples, comme l'indique le tableau 5.3 : deux mesurent la **génération**, deux mesurent la **recherche**. Cette séparation est précieuse en pratique, car elle permet d'attribuer une défaillance au bon composant.

Table 5.3 – Répartition des métriques RAGAS par composant évalué

Métrique	**Composant**	**Ce qu'un score faible indique**
Faithfulness	Génération	Le modèle invente : revoir l'invite système ou abaisser la température
Answer relevancy	Génération	Le modèle répond à côté : reformuler l'invite
Context precision	Recherche	Trop de bruit récupéré : réduire k, ajouter un réordonnancement
Context recall	Recherche	Information manquante : augmenter k, revoir le découpage ou le modèle d'*embeddings*

5.3.2 Mise en uvre

Conformément à la contrainte de confidentialité, le modèle juge est lui aussi exécuté localement : RAGAS est configuré pour utiliser le même Mistral 7B servi par Ollama, et le même modèle d'*embeddings*, plutôt que l'API OpenAI qu'il emploie par défaut.

Listing 5.1 – Configuration de RAGAS avec un juge local (`src/evaluation/ragas_eval.py`)

```
judge_llm = ChatOllama(
    base_url=settings.ollama_base_url,
    model=settings.ollama_model,
    temperature=0,
)
judge_embeddings = HuggingFaceEmbeddings(model_name=settings.embedding_model)

result = evaluate(
    dataset,
    metrics=[faithfulness, answer_relevancy, context_precision, context_recall],
    llm=judge_llm,
    embeddings=judge_embeddings,
)
```

Le jeu d'évaluation doit être fourni sous forme d'un fichier CSV à deux colonnes, `question` et `ground_truth`, cette dernière contenant la réponse de référence rédigée par un relecteur humain. Le module exécute alors le pipeline complet sur chaque question, assemble le quadruplet {question, réponse, contexte, référence} attendu par RAGAS, puis calcule les quatre métriques et enregistre le rapport dans `docs/ragas_report.json`.

Listing 5.2 – Lancement de l'évaluation

```
python -m src.evaluation.ragas_eval --testset data/eval/testset.csv
```

5.3.3 État d'avancement et résultats

Avertissement méthodologique. Le module d'évaluation est développé et opérationnel, mais la campagne de mesure RAGAS **n'a pas encore été exécutée** à la date de rédaction de ce rapport. Deux prérequis restent à lever : l'installation des dépendances `ragas` et `datasets` dans l'environnement, et surtout la **constitution du jeu d'évaluation annoté** — une vingtaine de questions accompagnées de leurs réponses de référence, rédigées manuellement à partir des documents du corpus. Le fichier `data/eval/testset.csv` ne contient à ce jour qu'un gabarit de trois questions non renseignées, dont le contenu porte de surcroît sur des documents absents du corpus (*cf.* section 3.3.2). Les valeurs du tableau 5.4 sont donc laissées à compléter ; elles ne sont pas estimées, afin de ne pas présenter comme mesuré ce qui ne l'a pas été.

Les valeurs cibles indiquées correspondent aux seuils usuellement retenus dans la littérature pour un système RAG considéré comme exploitable en production. Le seuil le plus exigeant porte sur la *faithfulness* : en contexte d'entreprise, une réponse sur dix contenant une affirmation inventée constitue déjà un niveau de risque difficilement acceptable.

Table 5.4 – Résultats RAGAS — à compléter après exécution de la campagne de mesure

Métrique	Score obtenu	Cible	Interprétation
Faithfulness	**à mesurer**	$> 0{,}90$	Taux d'ancrage documentaire ; métrique critique
Answer relevancy	**à mesurer**	$> 0{,}80$	Adéquation à la question posée
Context precision	**à mesurer**	$> 0{,}70$	Qualité du classement des extraits
Context recall	**à mesurer**	$> 0{,}70$	Complétude de la recherche

Compte tenu du diagnostic établi à la section précédente, une prévision peut néanmoins être formulée : les métriques de *recherche* (*context precision* et *context recall*) seront vraisemblablement les plus faibles, puisque nous avons établi expérimentalement que la recherche remonte fréquemment des documents inadaptés lorsque la question est posée en français. La *faithfulness* pourrait en revanche rester élevée : le modèle peut fort bien rester parfaitement fidèle à un contexte... qui n'est pas le bon. Ce cas de figure illustre précisément pourquoi une métrique unique serait trompeuse et pourquoi la décomposition en quatre dimensions est nécessaire.

5.4 Performance et latence

Le tableau 5.5 présente les mesures de performance relevées.

Table 5.5 – Mesures de performance du système

Étape	Durée	Remarque
Recherche vectorielle (top-$k = 4$)	182 ms	Moyenne sur 7 requêtes ; négligeable
Chargement du modèle d'*embeddings*	environ 2 s	Une seule fois, mis en cache
Génération de la réponse (CPU)	environ 110 s	Mesure relevée en conditions réelles
Ingestion complète du corpus	plusieurs minutes	2 929 segments, opération hors ligne

Le déséquilibre est frappant : la recherche représente moins de 0,2 % du temps de réponse total, la génération plus de 99 %. L'inférence d'un modèle de 7 milliards de paramètres sur CPU constitue l'unique goulot d'étranglement du système.

Cette latence d'environ deux minutes est acceptable en démonstration mais excessive pour un usage quotidien. Trois leviers permettraient de la réduire : l'exécution sur GPU, qui apporterait typiquement un facteur dix ; le recours à un modèle plus fortement quantifié (`mistral:7b-instruct-q4`) ou plus petit (`phi-3`, `llama3.2:3b`) ; ou la diffusion

progressive de la réponse (*streaming*), qui n'améliore pas le temps total mais réduit considérablement la latence perçue en affichant les premiers mots dès leur génération.

5.5 Leviers anti-hallucination : bilan

Le tableau 5.6 récapitule les mécanismes mis en place contre l'hallucination et leur état.

Table 5.6 – Leviers de réduction des hallucinations

Levier	État	Commentaire
Invite système restrictive	Implémenté	Interdit explicitement de sortir du contexte et fournit une formulation d'ignorance prête à l'emploi
Température basse (0,1)	Implémenté	Réduit la variabilité de génération et rend les réponses reproductibles
Garde-fou sur contexte vide	Implémenté	Court-circuite le modèle si la recherche ne remonte rien
Citation systématique des sources	Implémenté	Permet à l'utilisateur de vérifier chaque réponse ; levier le plus efficace en pratique
Affichage des scores de pertinence	Implémenté	Signale visuellement à l'utilisateur un appui documentaire faible
Seuil de score minimal	**à implémenter**	Écarté à ce stade : le tableau 5.1 montre qu'aucun seuil n'est exploitable avant le passage à un modèle multilingue
Ré-ordonnancement (*reranking*)	**à implémenter**	Un *cross-encoder* améliorerait la *context precision*
Mesure RAGAS	**à exécuter**	Le module est prêt ; le jeu annoté reste à constituer

Il convient de relever que le levier le plus efficace n'est pas d'ordre algorithmique. C'est la **citation systématique des sources** : en exposant les extraits utilisés et leur score, le système transforme une réponse invérifiable en une affirmation contrôlable. Même si le modèle hallucine, l'utilisateur dispose des moyens de s'en apercevoir. Cette transparence par construction est, en contexte professionnel, plus précieuse qu'un gain marginal de fidélité.

5.6 Conclusion

Ce chapitre a présenté l'évaluation du système à deux niveaux. L'évaluation quantitative de la recherche a établi un résultat majeur : la latence de recherche est négligeable (182 ms), mais les scores de pertinence sont anormalement faibles sur des questions françaises. Une expérience contrôlée a identifié la cause — l'asymétrie linguistique entre un

modèle d'*embeddings* anglophone et des questions françaises — et l'a quantifiée : la traduction de la question en anglais triple le score de pertinence. Cette mesure fonde la préconisation d'adopter un modèle multilingue. Une seconde limite structurelle a été documentée : l'incapacité de la recherche vectorielle à retrouver un identifiant exact, qui appelle une recherche hybride.

Le protocole d'évaluation RAGAS de la chaîne complète est développé et opérationnel, mais la campagne de mesure reste à conduire : elle requiert la constitution préalable d'un jeu de questions annotées. Les emplacements correspondants ont été laissés explicitement vacants plutôt que renseignés par estimation. Le chapitre suivant présente le déploiement du système.

CHAPITRE 6

Déploiement

6.1 Introduction

Ce chapitre présente la mise en service du système. Conformément au dernier objectif du stage, l'assistant est exposé par une API REST développée avec FastAPI, doté d'une interface conversationnelle réalisée avec Streamlit, et l'ensemble est conteneurisé avec Docker afin d'être reproductible sur toute machine hôte.

6.2 Architecture de déploiement

Le système est déployé en trois services distincts, comme l'illustre la figure 6.1. Cette décomposition n'est pas gratuite : elle traduit une décision d'architecture consistant à faire de l'API la **seule interface** entre le moteur RAG et le monde extérieur.

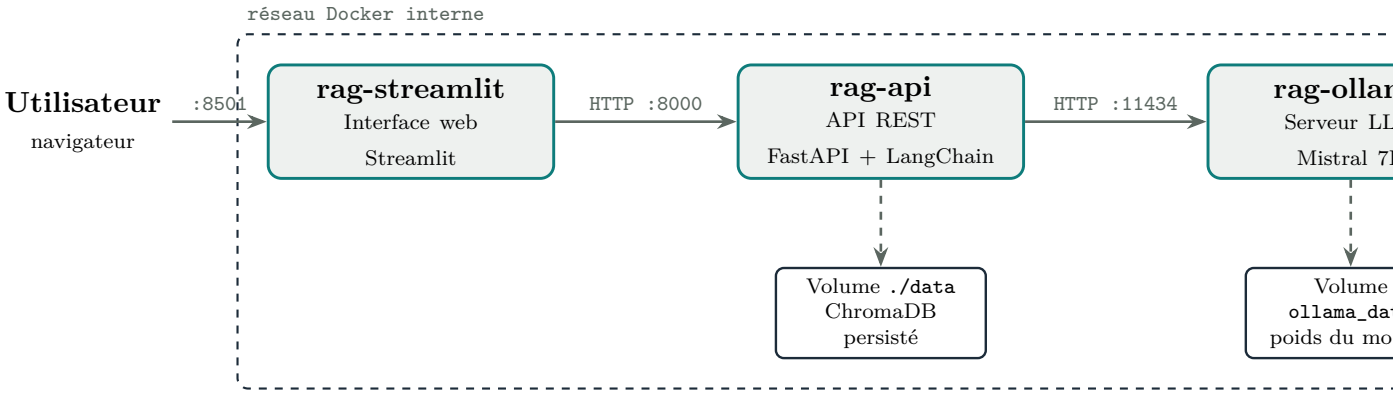


Figure 6.1 – Architecture de déploiement en trois conteneurs. Seul le port 8501 de l'interface est destiné aux utilisateurs ; l'API et le serveur LLM restent internes au réseau Docker.

L'intérêt de ce découplage est double. D'une part, l'interface Streamlit ne contient *aucune* logique RAG : elle se borne à émettre des requêtes HTTP. Il serait donc possible de la remplacer par un *bot* Slack, une extension Teams ou une application mobile sans toucher une ligne du moteur. D'autre part, l'API et le serveur de modèle peuvent être dimensionnés indépendamment — seul le service Ollama tire réellement profit d'un GPU.

6.3 L'API REST avec FastAPI

FastAPI a été retenu pour trois raisons : la validation automatique des données entrantes et sortantes via Pydantic, la génération automatique d'une documentation interactive OpenAPI, et sa nature asynchrone qui lui permet de soutenir plusieurs requêtes concurrentes.

Quatre *endpoints* sont exposés, décrits au tableau 6.1.

Table 6.1 – *Endpoints* de l'API REST

Méthode	Route	Description
`GET`	`/health`	Contrôle de disponibilité du service
`POST`	`/query`	Soumet une question, retourne la réponse, ses sources et leurs scores
`POST`	`/ingest`	Déclenche la (ré-)ingestion du corpus, avec option de réinitialisation
`GET`	`/collection/info`	Retourne l'état de l'index : nombre de segments, modèles utilisés

Les contrats d'entrée et de sortie sont déclarés par des modèles Pydantic, ce qui garantit qu'une requête malformée est rejetée avec un message explicite avant d'atteindre la logique métier :

Listing 6.1 – Contrats de l'*endpoint* de requête (`src/api/main.py`)

```python
class QueryRequest(BaseModel):
    question: str = Field(..., min_length=1)
    top_k: int | None = Field(default=None, ge=1, le=20)

class SourceItem(BaseModel):
    filename: str | None
    chunk_id: str | None
    relevance_score: float

class QueryResponse(BaseModel):
    answer: str
    sources: list[SourceItem]
    context_found: bool
```

Le champ `top_k` est borné entre 1 et 20 : cette contrainte empêche un client d'exiger un nombre d'extraits qui saturerait la fenêtre de contexte du modèle. Le champ `context_found` permet au client de distinguer une réponse fondée sur des documents d'un aveu d'ignorance — distinction exploitée par l'interface pour adapter son affichage.

Chaque *endpoint* encapsule ses erreurs et les traduit en codes HTTP appropriés, plutôt que de laisser remonter une trace d'exécution Python :

Listing 6.2 – Traitement des erreurs

```python
@app.post("/query", response_model=QueryResponse)
def query(req: QueryRequest) -> QueryResponse:
    try:
        result = answer_question(req.question, k=req.top_k)
    except Exception as exc:
        logger.exception("Query failed")
        raise HTTPException(status_code=500, detail=str(exc)) from exc
    ...
```

6.4 L'interface conversationnelle

L'interface a été développée avec Streamlit, qui permet de produire une application web complète en Python sans écrire de HTML ni de JavaScript. Elle communique exclusivement avec l'API par HTTP.

6.4.1 Écran d'accueil

La figure 6.2 présente l'état initial de l'application. La barre latérale expose en permanence l'état du système : disponibilité de l'API, nombre de segments indexés, modèle de langue et modèle d'*embeddings* actifs. Cette information est récupérée par appel à `/collection/info` et évite à l'utilisateur de s'interroger sur l'origine d'un dysfonctionnement.

La zone centrale propose des questions d'amorçage adaptées au contenu réel du corpus — produits d'une commande, montant d'une facture, stock d'un mois donné. Ce détail est important sur le plan de l'expérience utilisateur : sans elles, un utilisateur découvrant l'outil ignore ce qu'il peut légitimement demander, et une première question hors corpus produit une impression d'échec alors que le système fonctionne correctement.

6.4.2 Réponse sourcée

La figure 6.3 montre le système en fonctionnement sur une question réelle. La réponse est structurée, chaque bon de commande étant restitué avec son identifiant, sa date, le produit concerné, la quantité et le prix unitaire. Elle se termine par la liste explicite des fichiers sources utilisés.

Au-dessus de la réponse figure une ligne de métadonnées de traçabilité : le nombre d'extraits mobilisés, le meilleur score de pertinence obtenu et le temps de génération.

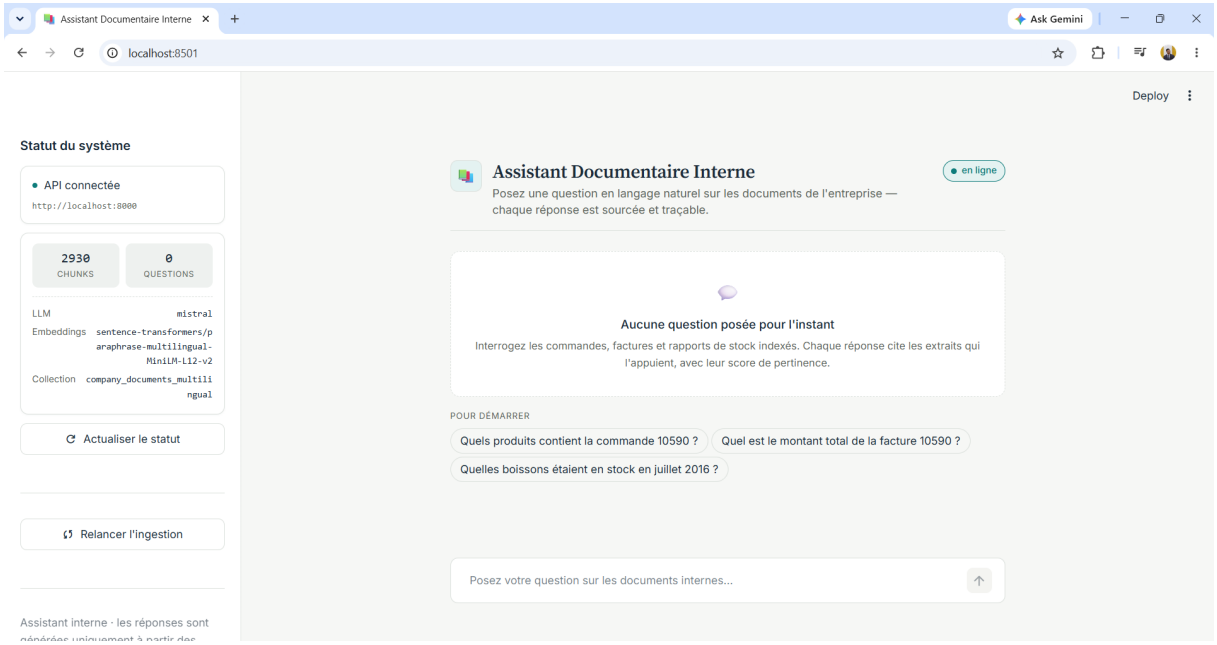


Figure 6.2 – Écran d'accueil de l'assistant. La barre latérale affiche l'état du système ; la zone centrale propose des questions d'amorçage cohérentes avec le corpus indexé.

Un panneau dépliable ń Provenance ż donne accès au détail de chaque extrait, avec son identifiant de segment et une barre indiquant visuellement son score.

Cet affichage constitue la traduction concrète du critère de traçabilité fixé au chapitre 2. L'utilisateur n'a pas à croire le système : il peut ouvrir les fichiers cités et vérifier. C'est ce qui distingue un assistant exploitable en entreprise d'une démonstration technique.

6.5 Conteneurisation avec Docker

6.5.1 Organisation

Le déploiement repose sur trois services orchestrés par `docker compose`, décrits au tableau 6.2.

Table 6.2 – Services définis dans `docker-compose.yml`

Service	Port	Image	Rôle
`rag-ollama`	11434	`ollama/ollama`	Sert le modèle Mistral 7B ; volume persistant pour les poids
`rag-api`	8000	`Dockerfile.api`	Moteur RAG et API REST ; monte `./data` pour accéder à l'index
`rag-streamlit`	8501	`Dockerfile.streamlit`	Interface web ; ne connaît que l'URL de l'API

Deux volumes assurent la persistance : `ollama_data` conserve les poids du modèle,

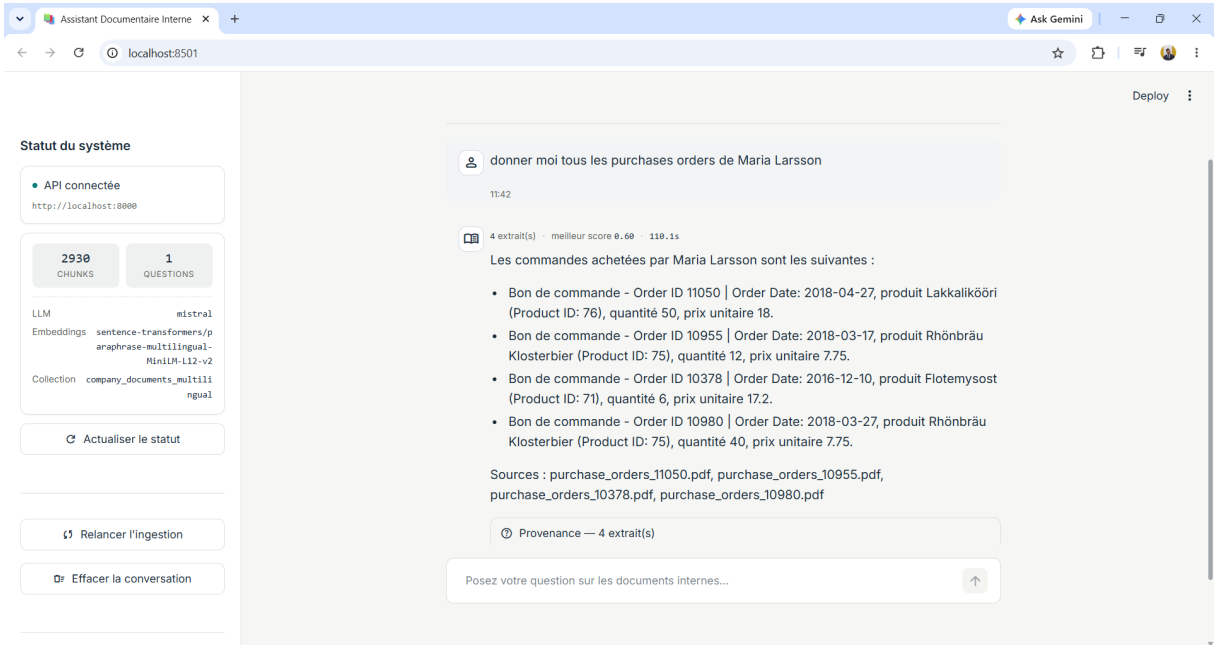


Figure 6.3 – Réponse sourcée produite par le système. Les quatre bons de commande sont restitués avec leurs détails, suivis de la liste des fichiers sources. La ligne de métadonnées indique le nombre d'extraits, le meilleur score et le temps de génération.

évitant un téléchargement de 4,4 Go à chaque redémarrage, et `./data` conserve l'index ChromaDB, évitant une ré-ingestion complète.

La configuration inter-services s'effectue par variables d'environnement, les conteneurs se résolvant par leur nom de service sur le réseau Docker interne :

Listing 6.3 – Extrait de `docker-compose.yml`

```
api:
  build: { context: ., dockerfile: Dockerfile.api }
  container_name: rag-api
  ports: ["8000:8000"]
  environment:
    - OLLAMA_BASE_URL=http://ollama:11434
    - CHROMA_PERSIST_DIR=/app/data/processed/chroma_db
  volumes: ["./data:/app/data"]
  depends_on: [ollama]
```

6.5.2 Mise en service

Le déploiement complet tient en deux commandes :

Listing 6.4 – Déploiement conteneurisé

```
docker compose up --build -d
docker exec -it rag-ollama ollama pull mistral
```

La seconde commande télécharge les poids du modèle dans le conteneur Ollama. Elle n'est nécessaire qu'au premier démarrage, le volume persistant conservant ensuite le modèle.

L'ingestion du corpus s'effectue alors par simple appel à l'API :

Listing 6.5 – Ingestion via l'API

```
curl -X POST http://localhost:8000/ingest \
     -H "Content-Type: application/json" \
     -d '{"reset": true}'
```

6.6 Points de vigilance avant mise en production

L'état actuel du déploiement est fonctionnel en environnement de développement, mais plusieurs points doivent être traités avant toute exposition réelle. Ils sont recensés au tableau 6.3 par ordre de criticité.

Table 6.3 – Points de vigilance identifiés

Point	Criticité	Action requise
Dépendances non figées	**Bloquant**	Plusieurs modules importés par le code (`pypdf`, `python-docx`, `sentence-transformers`, `httpx`, `ragas`, `datasets`) ne figurent pas dans `requirements.txt`. La construction des images Docker produit donc des conteneurs incomplets. Ce fichier doit être complété et les versions figées
Absence d'authentification	Élevée	L'API et l'interface n'ont aucun contrôle d'accès. Ajouter une clé d'API ou OAuth, et placer un *reverse proxy* avec HTTPS devant l'interface
Exposition des services internes	Élevée	Restreindre l'accès réseau à Ollama et ChromaDB, qui ne doivent pas être joignables publiquement
Journalisation d'audit	Moyenne	Tracer qui a posé quelle question et quelles sources ont été mobilisées, à des fins de conformité
Filtrage des documents sensibles	Moyenne	Anonymiser ou exclure les documents à caractère personnel avant ingestion (RGPD)
Latence de génération	Moyenne	Prévoir un GPU ou un modèle quantifié pour un usage quotidien (*cf.* chapitre 5)

Le premier point mérite un commentaire. Il illustre un écueil classique du développement : l'environnement local a été enrichi progressivement par installations successives, sans que le fichier de dépendances soit tenu à jour. Le code fonctionne donc parfaitement sur la machine de développement tout en étant irreproductible ailleurs — ce qui

est précisément ce que la conteneurisation est censée éliminer. La leçon est de valider systématiquement une construction depuis un environnement vierge, et non depuis l'environnement de travail.

6.7 Versionnement

Le code source est versionné avec Git. L'arborescence du projet sépare explicitement les responsabilités, comme le montre le listing 6.6.

Listing 6.6 – Organisation du dépôt

```
rag-enterprise-assistant/
|-- src/
|   |-- config.py              # configuration centralisee (.env)
|   |-- ingestion/             # chargement, decoupage, indexation
|   |-- rag/                   # recherche, generation, pipeline
|   |-- evaluation/            # metriques RAGAS
|   |-- api/                   # backend FastAPI
|   +-- app/                   # interface Streamlit
|-- scripts/ingest_documents.py
|-- data/
|   |-- raw/                   # corpus source
|   |-- processed/             # index ChromaDB persiste
|   +-- eval/testset.csv       # jeu d'evaluation
|-- tests/
|-- docs/                      # architecture, guide de deploiement
|-- Dockerfile.api
|-- Dockerfile.streamlit
|-- docker-compose.yml
|-- Makefile
+-- requirements.txt
```

Un `Makefile` centralise les opérations courantes (`make ingest`, `make api`, `make ui`, `make eval`, `make test`), ce qui réduit le risque d'erreur de manipulation et documente de fait les commandes utiles du projet.

6.8 Conclusion

Ce chapitre a présenté le déploiement du système en trois services conteneurisés : un serveur de modèle Ollama, une API REST FastAPI exposant quatre *endpoints*, et une interface conversationnelle Streamlit. Le découplage strict entre l'interface et le moteur permet de substituer l'une sans toucher à l'autre. Les captures présentées attestent du fonctionnement effectif de la chaîne complète, de la question posée jusqu'à la réponse

sourcée accompagnée de ses extraits et de leurs scores. Les points de vigilance à traiter avant une exposition réelle ont été recensés, le plus critique étant la complétion du fichier de dépendances.

Conclusion Générale

Ce stage a été consacré à la conception, à la réalisation et au déploiement d'un système RAG (*Retrieval-Augmented Generation*) permettant d'interroger en langage naturel un corpus de documents d'entreprise, avec des réponses sourcées et vérifiables. Le travail a été conduit selon la méthodologie CRISP-DM, dont chaque phase a été traitée successivement : compréhension du besoin métier, exploration et préparation d'un corpus de 2 676 documents, construction de la chaîne de traitement, évaluation de sa fiabilité et déploiement conteneurisé.

Bilan du travail réalisé

Sur le plan fonctionnel, la chaîne complète est opérationnelle. Le pipeline d'ingestion traite les cinq formats documentaires courants, extrait le texte, l'enrichit de métadonnées de traçabilité et le découpe en segments : les 2 676 fichiers du corpus ont été intégralement traités, sans aucun échec d'extraction, produisant 2 929 segments vectorisés et persistés dans ChromaDB. La recherche sémantique s'exécute en 182 millisecondes en moyenne. La génération est assurée localement par Mistral 7B, contraint par une invite qui lui interdit de sortir du contexte fourni. L'ensemble est exposé par une API REST à quatre *endpoints* et une interface conversationnelle, déployés en trois conteneurs Docker.

La contrainte structurante du projet a été respectée sans concession : **aucune donnée ne quitte l'infrastructure**. Vectorisation, indexation, recherche, génération et même l'évaluation par modèle juge s'exécutent localement, sans clé d'API ni coût récurrent. Cette propriété, qui aurait pu paraître une contrainte appauvrissante, s'est révélée le principal argument de l'architecture retenue face aux solutions du marché.

Sur le plan de la traçabilité, l'objectif est atteint. Chaque réponse expose les fichiers sources mobilisés, l'identifiant précis de chaque segment et son score de pertinence. L'utilisateur peut vérifier ce que le système avance plutôt que de devoir lui faire confiance — ce qui constitue, en contexte professionnel, la condition d'adoption d'un outil de ce type.

Principal apport de la démarche d'évaluation

Le résultat le plus instructif du stage n'est pas la mise en service du système, mais ce que sa mesure a révélé. À l'usage, l'assistant produisait des réponses d'apparence satisfaisante. L'évaluation quantitative de la composante de recherche a pourtant montré que les scores de pertinence étaient anormalement faibles sur des questions posées en français, et que le système s'appuyait fréquemment sur des documents inadaptés.

Une expérience contrôlée a permis d'en identifier et d'en quantifier la cause : à contenu sémantique identique, la traduction d'une question en anglais **triple le score de pertinence** (0,461 contre 0,153 en moyenne). Le modèle `all-MiniLM-L6-v2`, entraîné essentiellement sur de l'anglais, projette mal les questions françaises dans l'espace sémantique de documents anglophones.

Ce constat porte un enseignement méthodologique qui dépasse le cas particulier : **un système RAG peut paraître fonctionner tout en s'appuyant sur les mauvais documents**. Une réponse fluide, bien structurée et accompagnée de sources n'est pas pour autant une réponse correcte. Seule la mesure permet de distinguer les deux, et c'est ce qui justifie la place centrale que CRISP-DM accorde à la phase d'évaluation avant tout déploiement.

Limites du travail

Trois limites doivent être clairement énoncées.

La **campagne d'évaluation RAGAS n'a pas été menée à son terme**. Le module de mesure est développé et configuré pour fonctionner avec un modèle juge local, mais son exécution requiert la constitution préalable d'un jeu de questions annotées de réponses de référence rédigées manuellement. Les quatre métriques — *faithfulness*, *answer relevancy*, *context precision* et *context recall* — restent donc à mesurer. Le choix a été fait de laisser ces valeurs explicitement vacantes dans le présent rapport plutôt que de les estimer.

Le **corpus est exclusivement transactionnel**. Les 2 676 documents sont des factures, bons de commande, commandes et états de stock, c'est-à-dire des enregistrements structurés. Ils permettent de valider techniquement le pipeline, mais ils sous-exploitent l'intérêt propre d'une architecture RAG, qui donne son plein rendement sur des documents rédigés où l'information est diffuse. Sur des enregistrements structurés, une requête SQL serait plus exacte et plus rapide. Cette composition explique par ailleurs le caractère quasi inopérant du paramètre de découpage, la longueur moyenne d'un document (498 caractères) étant très inférieure à la taille de segment configurée (1 000 caractères).

La **latence de génération est excessive** pour un usage quotidien : environ 110 secondes par réponse. L'inférence d'un modèle de 7 milliards de paramètres sur CPU représente plus de 99 % du temps de réponse total, la recherche vectorielle étant quant à elle négligeable.

Perspectives

Les travaux futurs se hiérarchisent naturellement à partir des mesures obtenues.

Adopter un modèle d'*embeddings* multilingue. C'est l'action de rang le plus élevé, car la plus étayée expérimentalement. Le remplacement de `all-MiniLM-L6-v2` par `paraphrase-multilingual-MiniLM-L12-v2`, entraîné sur plus de cinquante langues, devrait résoudre l'asymétrie linguistique documentée au chapitre 5. L'opération impose une ré-indexation complète du corpus.

Conduire la campagne RAGAS. Une fois le modèle multilingue en place, la constitution d'un jeu de vingt questions annotées permettra d'établir une mesure de référence des quatre métriques, et surtout de comparer objectivement les configurations successives plutôt que de se fier à une impression d'usage.

Mettre en place une recherche hybride. La combinaison d'une composante lexicale exacte (BM25) et de la composante vectorielle, avec fusion des classements, lèverait la limite documentée sur la recherche par identifiant exact — besoin fréquent sur un corpus transactionnel où l'utilisateur connaît souvent le numéro de la pièce qu'il cherche.

Ajouter un ré-ordonnancement. L'insertion d'un *cross-encoder* après la recherche vectorielle, chargé de réévaluer finement la pertinence des extraits candidats, améliorerait la *context precision* et permettrait d'établir enfin un seuil de rejet fiable.

Enrichir le corpus de documents rédigés. L'ajout de procédures, manuels et notes d'organisation permettrait d'exercer le système sur le type de questions pour lequel une architecture RAG est réellement supérieure aux alternatives, et redonnerait au réglage du découpage son rôle de levier d'optimisation.

Réduire la latence. Le passage sur GPU, l'emploi d'un modèle plus fortement quantifié ou plus petit, et la diffusion progressive de la réponse constituent trois leviers cumulables.

En définitive, ce stage a permis de mener un projet de science des données de la formulation du besoin jusqu'à un système déployé et mesurable, en abordant l'ensemble de

la chaîne : ingénierie documentaire, recherche sémantique, ingénierie d'invite, évaluation de systèmes génératifs et déploiement conteneurisé. Au-delà des compétences techniques acquises, l'apport principal réside dans une conviction méthodologique : dans un système fondé sur un modèle génératif, la qualité perçue est un indicateur trompeur, et l'instrumentation de la mesure n'est pas une étape finale de validation mais la condition même du progrès.

Bibliographie et Nétographie

Articles scientifiques

[1] P. Lewis *et al.*, ń Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks ż, *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.
https://arxiv.org/abs/2005.11401

[2] N. Reimers et I. Gurevych, ń Sentence-BERT : Sentence Embeddings using Siamese BERT-Networks ż, *Proceedings of EMNLP-IJCNLP*, 2019.
https://arxiv.org/abs/1908.10084

[3] N. Reimers et I. Gurevych, ń Making Monolingual Sentence Embeddings Multilingual using Knowledge Distillation ż, *Proceedings of EMNLP*, 2020.
https://arxiv.org/abs/2004.09813

[4] S. Es, J. James, L. Espinosa-Anke et S. Schockaert, ń RAGAS : Automated Evaluation of Retrieval Augmented Generation ż, *Proceedings of EACL (System Demonstrations)*, 2024.
https://arxiv.org/abs/2309.15217

[5] A. Q. Jiang *et al.*, ń Mistral 7B ż, *arXiv preprint*, 2023.
https://arxiv.org/abs/2310.06825

[6] Y. A. Malkov et D. A. Yashunin, ń Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs ż, *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 42, nº 4, 2020.
https://arxiv.org/abs/1603.09320

[7] V. Karpukhin *et al.*, ń Dense Passage Retrieval for Open-Domain Question Answering ż, *Proceedings of EMNLP*, 2020.
https://arxiv.org/abs/2004.04906

[8] L. Huang *et al.*, ń A Survey on Hallucination in Large Language Models : Principles, Taxonomy, Challenges, and Open Questions ż, *ACM Transactions on Information Systems*, 2024.
https://arxiv.org/abs/2311.05232

[9] S. E. Robertson et H. Zaragoza, ń The Probabilistic Relevance Framework : BM25 and Beyond ż, *Foundations and Trends in Information Retrieval*, vol. 3, nº 4, 2009.

[10] R. Wirth et J. Hipp, ń CRISP-DM : Towards a Standard Process Model for Data Mi-

ning ż, *Proceedings of the 4th International Conference on the Practical Applications of Knowledge Discovery and Data Mining*, 2000.

Documentation technique

[11] **LangChain** — Documentation officielle du *framework* d'orchestration.
https://python.langchain.com/docs/

[12] **ChromaDB** — Documentation de la base de données vectorielle.
https://docs.trychroma.com/

[13] **Ollama** — Exécution locale de modèles de langue.
https://ollama.com/

[14] **Sentence-Transformers** — Bibliothèque de modèles d'*embeddings* de phrases.
https://www.sbert.net/

[15] **RAGAS** — *Framework* d'évaluation de systèmes RAG.
https://docs.ragas.io/

[16] **FastAPI** — *Framework* web asynchrone pour Python.
https://fastapi.tiangolo.com/

[17] **Streamlit** — Création d'applications web en Python.
https://docs.streamlit.io/

[18] **Docker Compose** — Orchestration de conteneurs multi-services.
https://docs.docker.com/compose/

[19] **pypdf** — Extraction de texte de fichiers PDF.
https://pypdf.readthedocs.io/

[20] **Pydantic** — Validation de données et gestion de configuration.
https://docs.pydantic.dev/

Ressources et jeux de données

[21] **Company Documents Dataset** — Corpus de documents d'entreprise utilisé dans ce projet, Kaggle.
https://www.kaggle.com/datasets/ayoubcherguelaine/company-documents-dataset

[22] **MTEB** (*Massive Text Embedding Benchmark*) — Classement comparatif des modèles d'*embeddings*, *Hugging Face*.
https://huggingface.co/spaces/mteb/leaderboard

[23] **all-MiniLM-L6-v2** — Fiche du modèle d'*embeddings* employé.
https://huggingface.co/sentence-transformers/all-MiniLM-L6-v2

[24] **paraphrase-multilingual-MiniLM-L12-v2** — Fiche du modèle multilingue préconisé en perspective.
https://huggingface.co/sentence-transformers/paraphrase-multilingual-MiniLM-L12-v2